PathAlgos

July 3, 2026

1 Terrain-Aware UAV Route Planning under Radar Threats using D* Lite

1.1 Research Objective

This notebook presents a modular implementation of **terrain-aware UAV route planning** in a radar-contested environment using the **D* Lite path planning algorithm**. The planner computes an optimal path over a two-dimensional grid where each cell represents the cumulative traversal cost obtained from multiple environmental factors.

The objective is to determine a safe, efficient, and feasible route between a specified **start** and **goal** location while minimizing the overall traversal cost.

---

2 Background

Modern UAV missions frequently operate in environments containing multiple hazards including:

- Complex terrain
- Radar surveillance systems
- No-fly zones
- Environmental penalties
- Restricted operational regions

Rather than planning solely based on geometric distance, this framework performs **cost-aware planning**, where every grid cell contains a weighted traversal cost representing mission risk.

The planner is based on the **D* Lite** incremental search algorithm, making it suitable for dynamic replanning and future extensions involving changing threat environments.

---

3 Input Data

The notebook expects a CSV file containing a **Final Cost Matrix**.

Each grid cell contains the combined traversal cost generated from factors such as:

- Terrain difficulty
- Radar threat intensity

- No-fly penalties
- Environmental penalties
- Additional mission-specific costs

Example:

`final_cost_matrix.csv`

---

4 Overall Workflow

The notebook is organized into the following stages:

1. Environment setup
2. Import required libraries
3. UAV database
4. Load Final Cost Matrix
5. Grid construction
6. D* Lite implementation
7. Route planning
8. Mission performance analysis
9. Threat analysis
10. Mission efficiency evaluation
11. Visualization
12. Results summary

---

5 Planning Algorithm

The path planner will implement **D* Lite entirely from scratch**.

Characteristics include:

- 2D grid environment
- 8-connected motion
- Cost-aware traversal
- Euclidean heuristic
- Incremental search structure
- Independent implementation (no external shortest-path libraries)

NetworkX will be used **only for visualization** and **not** for path planning.

---

6 UAV Integration

A UAV database will be integrated in subsequent sections using Python dataclasses.

Each UAV will provide mission-specific parameters such as:

- Cruise speed
- Fuel capacity
- Fuel consumption
- Maximum endurance
- Payload
- Maximum altitude
- Maximum climb angle

The planner will automatically compute mission metrics based on the selected UAV.

---

7 Mission Metrics

After planning, the notebook will evaluate:

- Path length
- Travel distance
- Total traversal cost
- Flight time
- Fuel consumption
- Remaining fuel
- Threat exposure
- Mission success
- Mission efficiency score

These metrics are designed to support quantitative comparison of alternative missions.

---

8 Visualization Outputs

The notebook will generate publication-quality figures including:

1. Final Cost Matrix heatmap
2. Grid graph with optimal D* Lite path
3. Cost map with planned trajectory
4. Mission statistics dashboard
5. Threat exposure histogram
6. Traversal cost profile
7. Cumulative mission cost
8. Fuel consumption profile

---

9 Assumptions

Throughout this implementation, the following assumptions are adopted:

- The environment is represented as a regular 2D grid.
- Each grid cell contains a precomputed traversal cost.

- UAV motion is allowed in eight directions.
- Cell traversal cost represents the combined environmental risk.
- Fuel consumption initially follows a linear distance-based model.
- Dynamic replanning capabilities of D* Lite can be extended in future work.

---

10 Expected Outputs

Upon execution, the notebook will produce:

- Optimal waypoint sequence
- Minimum-cost path
- Total traversal cost
- Distance traveled
- Mission duration
- Fuel usage
- Remaining fuel
- Threat exposure statistics
- Mission efficiency score
- Publication-quality visualizations
- Complete mission summary

Note: This notebook is designed as a modular research framework, allowing future integration of advanced UAV dynamics, adaptive fuel models, dynamic obstacle updates, moving radar systems, and real-time mission replanning.

[89]:
```
# Most libraries are pre-installed in Google Colab.
# The following command ensures required packages are available.

!pip -q install numpy pandas matplotlib networkx scipy

print("All required packages are installed successfully.")
```

```
All required packages are installed successfully.
```

[50]:
```
# Standard Library Imports


import math
import heapq
import time
from dataclasses import dataclass, field
from typing import Dict, List, Tuple, Optional, Set


# Numerical Computing
```

```
import numpy as np
import pandas as pd


# Scientific Computing


from scipy.spatial.distance import euclidean

# Graph Visualization
# (Used ONLY for visualization, NOT for path planning)


import networkx as nx

# Plotting


import matplotlib.pyplot as plt
from matplotlib.colors import Normalize


# Optional Colab Display Utilities


from IPython.display import display

print("Libraries imported successfully.")
```

```
Libraries imported successfully.
```

[51]:
```
# Global Plot Configuration
# Publication-Quality Figures for Research


plt.rcParams.update({

    # Figure
    "figure.figsize": (10, 8),
    "figure.dpi": 150,
    "savefig.dpi": 300,
    "savefig.bbox": "tight",

    # Fonts
    "font.family": "serif",
```

```
    "font.size": 12,
    "axes.labelsize": 13,
    "axes.titlesize": 15,
    "axes.titleweight": "bold",
    "legend.fontsize": 11,

    # Axes
    "axes.grid": True,
    "grid.linestyle": "--",
    "grid.alpha": 0.4,

    # Lines
    "lines.linewidth": 2.0,
    "lines.markersize": 6,

    # Ticks
    "xtick.labelsize": 11,
    "ytick.labelsize": 11,

    # Legend
    "legend.frameon": True,
    "legend.framealpha": 0.95,

    # Image Display
    "image.cmap": "jet",

    # Math Text
    "mathtext.fontset": "dejavuserif"
})

print("Publication-quality plotting configuration loaded.")
```

```
Publication-quality plotting configuration loaded.
```

[52]:
```
# DroneState Dataclass

# This dataclass stores the mission-relevant parameters of each
# UAV. These parameters will be used throughout the notebook for
# mission planning, fuel estimation, flight time calculation,
# and future performance models.

from dataclasses import dataclass
from typing import Optional


@dataclass
class DroneState:
```

```
    """Represents the specifications of a UAV."""

    # Identification
    name: str

    # Flight Performance
    max_speed: float                # km/h
    cruise_speed: float             # km/h

    # Fuel / Energy
    fuel_capacity: float            # liters (0 for battery-powered UAVs)
    fuel_consumption_rate: float    # liters/km (or battery units/km)
    battery_capacity: Optional[float]  # Wh (None for fuel-powered UAVs)

    # Mission Capability
    max_range: float                # km
    endurance: float                # hours
    payload: float                  # kg
    max_altitude: float             # meters

    # Physical Characteristics
    weight: float                   # kg
    wing_span: float                # meters

    # Additional Information
    remarks: str
```

[53]:
```
# Drone Database

# Stores a collection of UAVs that can be selected for mission
# planning. Additional UAVs can easily be added in the future.



class DroneDatabase:
    """Database containing supported UAV platforms."""

    def __init__(self):

        self._database = {

            "MQ-9 Reaper": DroneState(
                name="MQ-9 Reaper",
                max_speed=482,
                cruise_speed=313,
                fuel_capacity=1800,
                fuel_consumption_rate=1.05,
```

```
            battery_capacity=None,
            max_range=1850,
            endurance=27,
            payload=1700,
            max_altitude=15240,
            weight=4760,
            wing_span=20.1,
            remarks="Long-endurance MALE combat UAV"
        ),

        "Heron": DroneState(
            name="Heron",
            max_speed=207,
            cruise_speed=140,
            fuel_capacity=700,
            fuel_consumption_rate=0.45,
            battery_capacity=None,
            max_range=1000,
            endurance=45,
            payload=250,
            max_altitude=10000,
            weight=1150,
            wing_span=16.6,
            remarks="Medium Altitude Long Endurance UAV"
        ),

        "RQ-4 Global Hawk": DroneState(
            name="RQ-4 Global Hawk",
            max_speed=629,
            cruise_speed=575,
            fuel_capacity=7850,
            fuel_consumption_rate=2.25,
            battery_capacity=None,
            max_range=22780,
            endurance=34,
            payload=1360,
            max_altitude=18000,
            weight=14628,
            wing_span=39.9,
            remarks="High Altitude Long Endurance surveillance UAV"
        ),

        "Bayraktar TB2": DroneState(
            name="Bayraktar TB2",
            max_speed=220,
            cruise_speed=130,
            fuel_capacity=300,
```

```
            fuel_consumption_rate=0.28,
            battery_capacity=None,
            max_range=300,
            endurance=27,
            payload=150,
            max_altitude=8200,
            weight=700,
            wing_span=12.0,
            remarks="Tactical MALE UAV"
        ),

        "Ghatak": DroneState(
            name="Ghatak",
            max_speed=950,
            cruise_speed=700,
            fuel_capacity=3200,
            fuel_consumption_rate=1.75,
            battery_capacity=None,
            max_range=1500,
            endurance=6,
            payload=2000,
            max_altitude=12000,
            weight=8000,
            wing_span=12.0,
            remarks="Stealth UCAV under development"
        ),

        "Netra": DroneState(
            name="Netra",
            max_speed=35,
            cruise_speed=25,
            fuel_capacity=0,
            fuel_consumption_rate=0.04,
            battery_capacity=450,
            max_range=10,
            endurance=0.75,
            payload=0.3,
            max_altitude=300,
            weight=1.5,
            wing_span=0.9,
            remarks="Battery-powered quadrotor reconnaissance UAV"
        )

    }


# List all UAV names
```

```
    def list_drones(self):
        return list(self._database.keys())

    # Retrieve a UAV by name

    def get_drone(self, name: str) -> DroneState:

        if name not in self._database:
            raise KeyError(f"Drone '{name}' not found in database.")

        return self._database[name]


# Instantiate Database


drone_database = DroneDatabase()

print(f"Loaded {len(drone_database.list_drones())} UAVs.")
```

```
Loaded 6 UAVs.
```

[54]:
```
# Display Available UAVs

# Convert the UAV database into a Pandas DataFrame for easy
# inspection and comparison.


drone_records = []

for drone_name in drone_database.list_drones():

    drone = drone_database.get_drone(drone_name)

    drone_records.append({
        "Name": drone.name,
        "Cruise Speed (km/h)": drone.cruise_speed,
        "Max Speed (km/h)": drone.max_speed,
        "Range (km)": drone.max_range,
        "Endurance (hr)": drone.endurance,
        "Fuel Capacity": drone.fuel_capacity,
        "Battery Capacity (Wh)": drone.battery_capacity,
        "Payload (kg)": drone.payload,
        "Max Altitude (m)": drone.max_altitude,
        "Weight (kg)": drone.weight,
        "Wing Span (m)": drone.wing_span,
```

```
        "Remarks": drone.remarks
    })

drone_df = pd.DataFrame(drone_records)

# Improve readability
pd.set_option("display.max_columns", None)
pd.set_option("display.width", None)
pd.set_option("display.precision", 2)

print("=" * 90)
print("Available UAV Platforms")
print("=" * 90)

display(drone_df)
```

```
==========================================================================================
==========
Available UAV Platforms
==========================================================================================
==========

               Name  Cruise Speed (km/h)  Max Speed (km/h)  Range (km)  \
0       MQ-9 Reaper                  313               482        1850
1             Heron                  140               207        1000
2  RQ-4 Global Hawk                  575               629       22780
3     Bayraktar TB2                  130               220         300
4            Ghatak                  700               950        1500
5             Netra                   25                35          10

   Endurance (hr)  Fuel Capacity  Battery Capacity (Wh)  Payload (kg)  \
0           27.00           1800                    NaN        1700.0
1           45.00            700                    NaN         250.0
2           34.00           7850                    NaN        1360.0
3           27.00            300                    NaN         150.0
4            6.00           3200                    NaN        2000.0
5            0.75              0                  450.0           0.3

   Max Altitude (m)  Weight (kg)  Wing Span (m)  \
0             15240       4760.0           20.1
1             10000       1150.0           16.6
2             18000      14628.0           39.9
3              8200        700.0           12.0
4             12000       8000.0           12.0
5               300          1.5            0.9

                                  Remarks
0                Long-endurance MALE combat UAV
```

```
1             Medium Altitude Long Endurance UAV
2  High Altitude Long Endurance surveillance UAV
3                              Tactical MALE UAV
4                 Stealth UCAV under development
5   Battery-powered quadrotor reconnaissance UAV
```

[55]:
```python
# Upload Final Cost Matrix CSV

# Upload the CSV file containing the final traversal cost matrix.
# The uploaded file should be named:
#
#     final_cost.csv
#
# Each cell represents the cumulative traversal cost after
# combining terrain, radar threat, no-fly penalties, and any
# additional environmental costs.


from google.colab import files

print("Please upload 'final_cost.csv'...")

uploaded = files.upload()

# Retrieve uploaded filename
csv_filename = next(iter(uploaded.keys()))

print(f"\nSuccessfully uploaded: {csv_filename}")
```

```
Please upload 'final_cost.csv'…
```

```
<IPython.core.display.HTML object>
```

```
Saving final_cost.csv to final_cost (1).csv

Successfully uploaded: final_cost (1).csv
```

[56]:
```python
# CELL 9: Load and Validate the Final Cost Matrix

# This cell:
#   1. Loads the uploaded CSV file.
#   2. Converts it into a NumPy array.
#   3. Validates the matrix.
#   4. Identifies obstacle/no-fly cells.
#   5. Displays useful statistics for the planning environment.
#
# Assumptions:
#   - Cost = 0           : Free or safest traversable cell
```

```
#   - Cost > 0           : Traversable with associated cost
#   - Cost >= 999999     : Impassable obstacle / No-fly region

# Load CSV


cost_df = pd.read_csv(csv_filename, header=None)

# Convert to NumPy array
cost_matrix = cost_df.to_numpy(dtype=float)


# Validation


# Check for missing values
if np.isnan(cost_matrix).any():
    raise ValueError("Validation Error: Cost matrix contains NaN values.")

# Matrix dimensions
rows, cols = cost_matrix.shape

# Check for square matrix
if rows != cols:
    raise ValueError(
        f"Validation Error: Expected a square matrix but received "
        f"{rows} × {cols}."
    )

# Negative costs are not permitted
if np.any(cost_matrix < 0):
    raise ValueError(
        "Validation Error: Negative traversal costs are not allowed."
    )


# Define Obstacle Threshold


OBSTACLE_COST = 999999

# Boolean obstacle mask
obstacle_mask = cost_matrix >= OBSTACLE_COST


# Store Grid Information
```

```python
GRID_ROWS = rows
GRID_COLS = cols

# Display Statistics


print("=" * 65)
print("        FINAL COST MATRIX SUCCESSFULLY LOADED")
print("=" * 65)

print(f"Grid Size                : {GRID_ROWS} × {GRID_COLS}")
print(f"Data Type                : {cost_matrix.dtype}")

print("\nCost Statistics")
print("-" * 65)
print(f"Minimum Traversal Cost   : {cost_matrix.min():.6f}")
print(f"Maximum Traversal Cost   : {cost_matrix.max():.2f}")
print(f"Mean Traversal Cost      : {cost_matrix.mean():.6f}")

print("\nEnvironment Statistics")
print("-" * 65)
print(f"Traversable Cells        : {np.sum(~obstacle_mask):,}")
print(f"Obstacle Cells           : {np.sum(obstacle_mask):,}")
print(f"Zero-Cost Cells          : {np.sum(cost_matrix == 0):,}")

print("\nValidation Status")
print("-" * 65)
print("  No missing (NaN) values")
print("  Square cost matrix")
print("  No negative traversal costs")
print("  Obstacle mask generated successfully")

print("=" * 65)
```

```
=================================================================
        FINAL COST MATRIX SUCCESSFULLY LOADED
=================================================================
Grid Size                : 100 × 100
Data Type                : float64

Cost Statistics
-----------------------------------------------------------------
Minimum Traversal Cost   : 0.000000
Maximum Traversal Cost   : 999999.00
Mean Traversal Cost      : 131899.883047
```

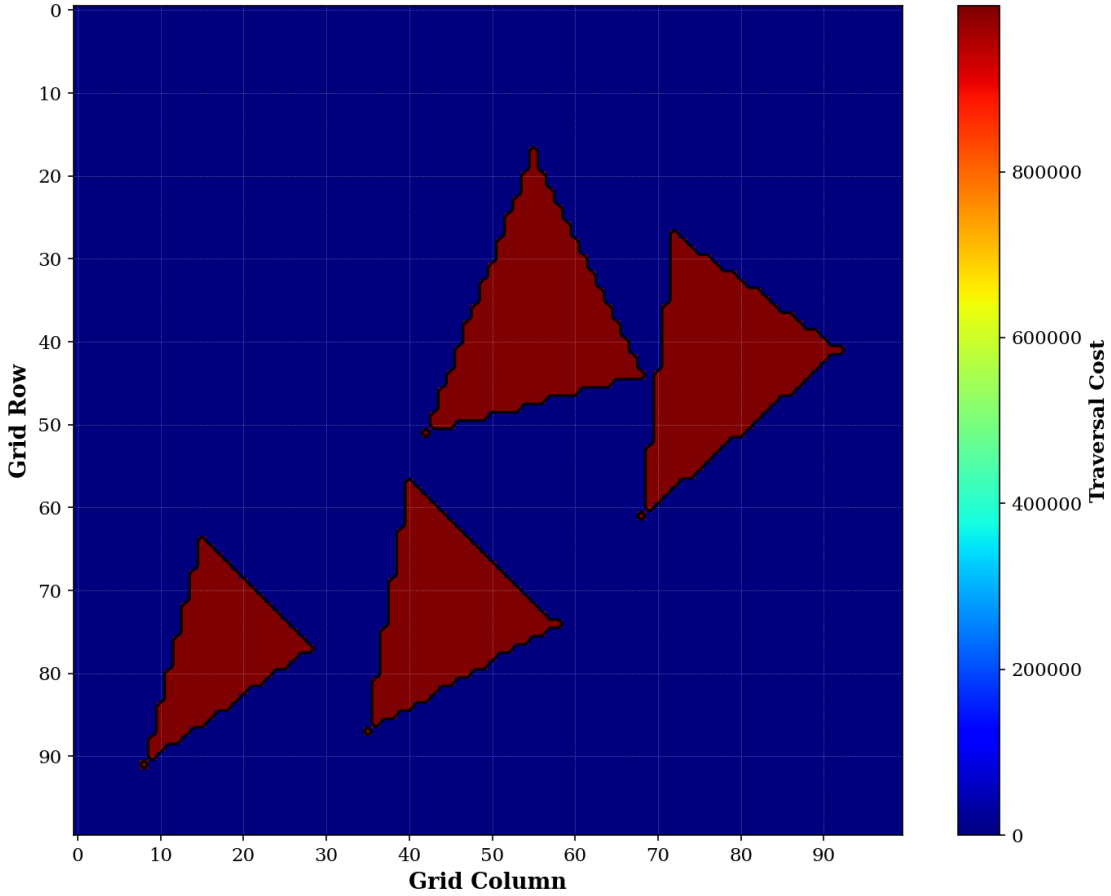
```
Environment Statistics
----------------------------------------------------------------
Traversable Cells        : 8,681
Obstacle Cells           : 1,319
Zero-Cost Cells          : 6,437

Validation Status
----------------------------------------------------------------
 No missing (NaN) values
 Square cost matrix
 No negative traversal costs
 Obstacle mask generated successfully
================================================================
```

[57]:
```python
# CELL 10: Figure 1 - Final Cost Matrix Heatmap

# This figure visualizes the combined traversal cost matrix used
# for UAV route planning.
#
# Visualization Features:
#   • Jet colormap
#   • Publication-quality formatting
#   • Colorbar indicating traversal cost
#   • Obstacle / No-Fly region overlay
#   • Grid coordinates
#
# Obstacle cells are defined as:
#       cost >= OBSTACLE_COST


# Create figure
fig, ax = plt.subplots(figsize=(10, 8))


# Display Cost Matrix


im = ax.imshow(
    cost_matrix,
    cmap="jet",
    origin="upper",
    interpolation="nearest"
)


# Colorbar
```

```
cbar = plt.colorbar(im, ax=ax)
cbar.set_label(
    "Traversal Cost",
    fontsize=12,
    fontweight="bold"
)


# Highlight Obstacle Regions


# Draw black contours around obstacle cells
ax.contour(
    obstacle_mask,
    levels=[0.5],
    colors="black",
    linewidths=1.5
)


# Labels and Title


ax.set_title(
    "Figure 1. Final Traversal Cost Matrix",
    fontsize=16,
    fontweight="bold",
    pad=15
)

ax.set_xlabel(
    "Grid Column",
    fontsize=13,
    fontweight="bold"
)

ax.set_ylabel(
    "Grid Row",
    fontsize=13,
    fontweight="bold"
)


# Tick Configuration
```

```
ax.set_xticks(np.arange(0, GRID_COLS, max(1, GRID_COLS // 10)))
ax.set_yticks(np.arange(0, GRID_ROWS, max(1, GRID_ROWS // 10)))

# Light grid for easier interpretation
ax.grid(
    which="major",
    color="white",
    linestyle=":",
    linewidth=0.4,
    alpha=0.5
)


# Layout


plt.tight_layout()


# Display Figure


plt.show()
```

Figure 1. Final Traversal Cost Matrix



[58]:
```
# CELL 11: Grid Configuration

# This cell defines the movement model used by the planner.
#
# Features
# --------
# • 8-connected grid
# • Cardinal and diagonal motion
# • Configurable movement costs
# • Coordinate type definitions
#
# The D* Lite planner developed later will use this motion model
# for graph expansion.


from typing import Tuple, List
```

```
# Type Alias


Coordinate = Tuple[int, int]


# Movement Cost Configuration

# These values can be modified later to reflect different
# vehicle dynamics or terrain traversal models.


CARDINAL_MOVE_COST: float = 1.0
DIAGONAL_MOVE_COST: float = np.sqrt(2)


# 8-Connected Motion Model
#
# Format:
# (row_offset, column_offset, movement_distance)


MOTION_MODEL: List[Tuple[int, int, float]] = [

    # Cardinal Directions
    (-1,  0, CARDINAL_MOVE_COST),   # North
    ( 1,  0, CARDINAL_MOVE_COST),   # South
    ( 0, -1, CARDINAL_MOVE_COST),   # West
    ( 0,  1, CARDINAL_MOVE_COST),   # East

    # Diagonal Directions
    (-1, -1, DIAGONAL_MOVE_COST),   # North-West
    (-1,  1, DIAGONAL_MOVE_COST),   # North-East
    ( 1, -1, DIAGONAL_MOVE_COST),   # South-West
    ( 1,  1, DIAGONAL_MOVE_COST)    # South-East
]

print("Grid configuration initialized.")
print(f"Connectivity : {len(MOTION_MODEL)}")
```

```
Grid configuration initialized.
Connectivity : 8
```

[59]:
```
# CELL 12: Weighted Grid Environment

# This class encapsulates the planning environment.
```

```
#
# Responsibilities

# • Grid validation
# • Obstacle checking
# • Neighbor generation
# • Movement cost computation
# • Euclidean distance calculation
#
# The D* Lite planner will interact exclusively with this class.


class WeightedGrid:
    """
    Represents a weighted 2D planning environment.
    """

    def __init__(
        self,
        cost_matrix: np.ndarray,
        obstacle_mask: np.ndarray,
        obstacle_cost: float = OBSTACLE_COST
    ) -> None:

        self.cost_matrix = cost_matrix
        self.obstacle_mask = obstacle_mask

        self.rows, self.cols = cost_matrix.shape

        self.obstacle_cost = obstacle_cost


    # Check whether a cell is inside the grid


    def is_valid(self, node: Coordinate) -> bool:
        """
        Returns True if the coordinate lies inside the grid and
        is not an obstacle.
        """

        r, c = node

        if r < 0 or r >= self.rows:
            return False

        if c < 0 or c >= self.cols:
```

```
            return False

        if self.obstacle_mask[r, c]:
            return False

        return True

    # Return valid neighboring cells


    def neighbors(self, node: Coordinate) -> List[Coordinate]:
        """
        Returns all valid neighboring cells using 8-connectivity.
        """

        neighbors = []

        r, c = node

        for dr, dc, _ in MOTION_MODEL:

            nr = r + dr
            nc = c + dc

            if self.is_valid((nr, nc)):
                neighbors.append((nr, nc))

        return neighbors


    # Traversal Cost


    def movement_cost(
        self,
        current: Coordinate,
        neighbor: Coordinate
    ) -> float:
        """
        Computes traversal cost between two adjacent cells.

        Cost =
            movement distance × destination cell cost
        """

        dr = abs(current[0] - neighbor[0])
        dc = abs(current[1] - neighbor[1])
```

```
        if dr == 1 and dc == 1:
            movement_distance = DIAGONAL_MOVE_COST
        else:
            movement_distance = CARDINAL_MOVE_COST

        return movement_distance * self.cost_matrix[neighbor]


    # Euclidean Distance


    def distance(
        self,
        node1: Coordinate,
        node2: Coordinate
    ) -> float:
        """
        Euclidean distance between two cells.
        """

        return float(np.hypot(
            node1[0] - node2[0],
            node1[1] - node2[1]
        ))
```

[60]:
```
# CELL 13: Build Weighted Grid

# Construct the weighted planning environment from the loaded
# cost matrix.
#
# This object will be passed directly to the D* Lite planner in
# the following notebook section.


# Create planning environment
grid = WeightedGrid(
    cost_matrix=cost_matrix,
    obstacle_mask=obstacle_mask,
    obstacle_cost=OBSTACLE_COST
)


# Environment Summary


print("=" * 65)
```

```python
print("WEIGHTED GRID ENVIRONMENT")
print("=" * 65)

print(f"Grid Size              : {grid.rows} × {grid.cols}")
print(f"Connectivity           : 8")
print(f"Obstacle Threshold     : {OBSTACLE_COST}")
print(f"Traversable Cells      : {np.sum(~obstacle_mask):,}")
print(f"Obstacle Cells         : {np.sum(obstacle_mask):,}")

print("=" * 65)


# Verify Neighbor Expansion


# Find a valid sample node near the center
sample = None
center_r = grid.rows // 2
center_c = grid.cols // 2

for r in range(center_r, grid.rows):
    found = False
    for c in range(center_c, grid.cols):
        if grid.is_valid((r, c)):
            sample = (r, c)
            found = True
            break
    if found:
        break

if sample is not None:
    print(f"\nSample Node           : {sample}")
    print(f"Valid Neighbors       : {len(grid.neighbors(sample))}")
    print(grid.neighbors(sample))
else:
    print("\nNo traversable cell found for neighbor verification.")
```

```
================================================================
WEIGHTED GRID ENVIRONMENT
================================================================
Grid Size              : 100 × 100
Connectivity           : 8
Obstacle Threshold     : 999999
Traversable Cells      : 8,681
Obstacle Cells         : 1,319
================================================================
```

```
Sample Node           : (50, 50)
Valid Neighbors       : 7
[(49, 50), (51, 50), (50, 49), (50, 51), (49, 51), (51, 49), (51, 51)]
```

[61]:
```python
# CELL 14: Priority Queue for D* Lite

#
# D* Lite maintains an OPEN list ordered by a two-component key:
#
#       key(s) = [k1, k2]
#
# where
#
#   k1 = min(g(s), rhs(s)) + h(start,s) + km
#   k2 = min(g(s), rhs(s))
#
# The OPEN list must efficiently support:
#
#   • Insert
#   • Update
#   • Remove
#   • Pop Minimum
#   • Membership Test
#
# This implementation uses Python's heapq together with
# lazy deletion for efficiency.
#
# Time Complexity

# Insert : O(log n)
# Remove : O(1) (lazy)
# Pop    : O(log n)


import heapq
from typing import Dict, List, Optional, Tuple

# Type Aliases


Priority = Tuple[float, float]
Coordinate = Tuple[int, int]


class PriorityQueue:
    """
    Priority Queue used by the D* Lite algorithm.
```

```
    Each entry consists of

        (priority_key, node)

    where

        priority_key = (k1, k2)

    Lazy deletion is employed to efficiently support key updates.
    """

    # Constructor


    def __init__(self) -> None:

        # Binary Heap
        self.heap: List[Tuple[Priority, Coordinate]] = []

        # Current valid entries
        self.entry_finder: Dict[Coordinate, Priority] = {}


    # Check whether queue is empty


    def empty(self) -> bool:
        """Return True if the OPEN list is empty."""
        return len(self.entry_finder) == 0


    # Insert / Update


    def push(
        self,
        node: Coordinate,
        priority: Priority
    ) -> None:
        """
        Insert a node or update its priority.

        Parameters
        ----------
        node : Coordinate
            Grid coordinate.
```

```
        priority : tuple(float,float)
            D* Lite priority key.
        """

        self.entry_finder[node] = priority

        heapq.heappush(
            self.heap,
            (priority, node)
        )

    # Remove


    def remove(
        self,
        node: Coordinate
    ) -> None:
        """
        Remove a node from the queue.

        Removal is lazy.

        The heap entry remains until encountered during pop().
        """

        if node in self.entry_finder:
            del self.entry_finder[node]

    # Membership


    def contains(
        self,
        node: Coordinate
    ) -> bool:
        """
        Check if a node is currently in OPEN.
        """

        return node in self.entry_finder


    # Peek Minimum
```

```
    def top_key(self) -> Priority:
        """
        Return the smallest key without removing it.
        """

        while self.heap:

            priority, node = self.heap[0]

            if (
                node in self.entry_finder
                and self.entry_finder[node] == priority
            ):
                return priority

            heapq.heappop(self.heap)

        return (float("inf"), float("inf"))


    # Pop Minimum


    def pop(self) -> Tuple[Priority, Coordinate]:
        """
        Remove and return the minimum-key node.
        """

        while self.heap:

            priority, node = heapq.heappop(self.heap)

            if (
                node in self.entry_finder
                and self.entry_finder[node] == priority
            ):

                del self.entry_finder[node]

                return priority, node

        raise KeyError("Priority Queue is empty.")


    # Queue Size
```

```
    def __len__(self) -> int:
        """
        Number of active nodes in OPEN.
        """

        return len(self.entry_finder)


    # Clear Queue


    def clear(self) -> None:
        """
        Remove all entries.
        """

        self.heap.clear()
        self.entry_finder.clear()

    # String Representation


    def __repr__(self) -> str:

        return (
            f"PriorityQueue("
            f"size={len(self)}, "
            f"heap_entries={len(self.heap)})"
        )


print("  D* Lite Priority Queue initialized successfully.")
```

```
 D* Lite Priority Queue initialized successfully.
```

[62]:
```
# CELL 15: Simplified D* Lite Planner (Prototype)

#
# This prototype implements a static grid planner inspired by
# D* Lite.
#
# Features
# --------
# • Uses the previously defined PriorityQueue
# • Works on the WeightedGrid
# • Supports 8-connected movement
# • Uses Euclidean heuristic
```

```
# • Returns
#       - Optimal path
#       - Total traversal cost
#       - Travel distance
#
# NOTE:
# This prototype does not implement dynamic replanning,
# rhs-values or km updates.
#


from typing import Dict, List, Tuple, Optional

Coordinate = Tuple[int, int]


class DStarLitePlanner:
    """
    Simplified D* Lite Planner for static environments.
    """

    def __init__(
        self,
        grid: WeightedGrid,
        start: Coordinate,
        goal: Coordinate
    ) -> None:

        self.grid = grid
        self.start = start
        self.goal = goal

        self.open_list = PriorityQueue()

        # Cost-to-come
        self.g_cost: Dict[Coordinate, float] = {
            start: 0.0
        }

        # Parent pointers
        self.parent: Dict[Coordinate, Optional[Coordinate]] = {
            start: None
        }


    # Heuristic
```

```
    def heuristic(
        self,
        node: Coordinate
    ) -> float:
        """
        Euclidean distance to goal.
        """

        return self.grid.distance(node, self.goal)


    # Priority Key


    def calculate_key(
        self,
        node: Coordinate
    ) -> Tuple[float, float]:

        g = self.g_cost.get(node, float("inf"))

        return (
            g + self.heuristic(node),
            g
        )

    # Plan


    def plan(
        self
    ) -> Tuple[List[Coordinate], float, float]:

        self.open_list.push(
            self.start,
            self.calculate_key(self.start)
        )

        visited = set()

        while not self.open_list.empty():

            _, current = self.open_list.pop()

            if current in visited:
                continue
```

```
            visited.add(current)

            if current == self.goal:
                break

            for neighbor in self.grid.neighbors(current):

                new_cost = (
                    self.g_cost[current] +
                    self.grid.movement_cost(current, neighbor)
                )

                if new_cost < self.g_cost.get(
                    neighbor,
                    float("inf")
                ):

                    self.g_cost[neighbor] = new_cost

                    self.parent[neighbor] = current

                    self.open_list.push(
                        neighbor,
                        self.calculate_key(neighbor)
                    )

        return self.reconstruct_path()

    # Path Reconstruction


    def reconstruct_path(
        self
    ) -> Tuple[List[Coordinate], float, float]:

        if self.goal not in self.parent:
            raise RuntimeError("No path found.")

        path = []

        node = self.goal

        while node is not None:

            path.append(node)
```

```
            node = self.parent[node]

        path.reverse()

        total_distance = 0.0
        total_cost = 0.0

        for i in range(len(path) - 1):

            total_distance += self.grid.distance(
                path[i],
                path[i + 1]
            )

            total_cost += self.grid.movement_cost(
                path[i],
                path[i + 1]
            )

        return path, total_distance, total_cost


print("  Simplified D* Lite Planner initialized successfully.")
```

```
 Simplified D* Lite Planner initialized successfully.
```

[63]:
```
# CELL 16: User Mission Configuration

#
# This cell allows the user to configure the mission by
# specifying:
#
#   • UAV Type
#   • Start Coordinate
#   • Goal Coordinate
#
# The selected UAV is automatically retrieved from the
# DroneDatabase.



# Display Available UAVs


print("=" * 60)
print("AVAILABLE UAVS")
print("=" * 60)
```

```python
for drone_name in drone_database.list_drones():
    print(f"• {drone_name}")

print("=" * 60)


# User Inputs


selected_drone = input(
    "\nEnter UAV Name: "
).strip()

start_row = int(input("Enter Start Row    : "))
start_col = int(input("Enter Start Column : "))

goal_row = int(input("Enter Goal Row     : "))
goal_col = int(input("Enter Goal Column  : "))

start = (start_row, start_col)
goal = (goal_row, goal_col)


# Validate Drone


if selected_drone not in drone_database.list_drones():
    raise ValueError(
        f"'{selected_drone}' is not available in the UAV database."
    )


# Retrieve Drone Information


drone = drone_database.get_drone(selected_drone)


# Validate Coordinates


if not grid.is_valid(start):
    raise ValueError(f"Invalid Start Coordinate: {start}")

if not grid.is_valid(goal):
    raise ValueError(f"Invalid Goal Coordinate: {goal}")
```

```python
# Display Mission Configuration


print("\n" + "=" * 60)
print("MISSION CONFIGURATION")
print("=" * 60)

print(f"Selected UAV      : {drone.name}")
print(f"Cruise Speed      : {drone.cruise_speed:.2f} m/s")
print(f"Maximum Speed     : {drone.max_speed:.2f} m/s")
print(f"Maximum Range     : {drone.max_range:.2f} km")
print(f"Fuel Capacity     : {drone.fuel_capacity:.2f}")
print(f"Endurance         : {drone.endurance:.2f} hr")
print(f"Payload           : {drone.payload:.2f} kg")

print(f"\nStart Coordinate  : {start}")
print(f"Goal Coordinate   : {goal}")

print("=" * 60)
```

```
============================================================
AVAILABLE UAVS
============================================================
• MQ-9 Reaper
• Heron
• RQ-4 Global Hawk
• Bayraktar TB2
• Ghatak
• Netra
============================================================

Enter UAV Name: MQ-9 Reaper
Enter Start Row    : 2
Enter Start Column : 26
Enter Goal Row     : 87
Enter Goal Column  : 26

============================================================
MISSION CONFIGURATION
============================================================
Selected UAV      : MQ-9 Reaper
Cruise Speed      : 313.00 m/s
Maximum Speed     : 482.00 m/s
Maximum Range     : 1850.00 km
Fuel Capacity     : 1800.00
```

```
Endurance          : 27.00 hr
Payload            : 1700.00 kg

Start Coordinate   : (2, 26)
Goal Coordinate    : (87, 26)
============================================================
```

[64]:
```python
# CELL 17: Execute D* Lite Path Planning

#
# This cell:
#   • Creates the planner
#   • Executes the planning algorithm
#   • Measures execution time
#   • Displays the planning results
#


import time

print("=" * 60)
print("EXECUTING D* LITE PATH PLANNING")
print("=" * 60)


# Start Timer


planning_start_time = time.perf_counter()


# Initialize Planner


planner = DStarLitePlanner(
    grid=grid,
    start=start,
    goal=goal
)


# Compute Optimal Path


try:

    path, total_distance, total_cost = planner.plan()
```

```
    planning_success = True

except RuntimeError as e:

    planning_success = False

    print("\nPlanning Failed!")
    print(e)


# Stop Timer


planning_end_time = time.perf_counter()

execution_time = planning_end_time - planning_start_time


# Display Results


if planning_success:

    print("\nPlanning Successful")
    print("-" * 60)

    print(f"Number of Waypoints : {len(path)}")
    print(f"Travel Distance     : {total_distance:.2f} grid units")
    print(f"Traversal Cost      : {total_cost:.4f}")
    print(f"Execution Time      : {execution_time:.6f} seconds")

    print("\nFirst 10 Waypoints")

    for i, waypoint in enumerate(path[:10]):
        print(f"{i+1:2d}. {waypoint}")

    if len(path) > 10:
        print("...")

else:

    print("\nNo feasible path exists between the selected Start and Goal.")
```

```
============================================================
EXECUTING D* LITE PATH PLANNING
============================================================
```

```
Planning Successful
------------------------------------------------------------
Number of Waypoints : 86
Travel Distance     : 87.49 grid units
Traversal Cost      : 4.0804
Execution Time      : 0.005077 seconds

First 10 Waypoints
 1. (2, 26)
 2. (3, 26)
 3. (4, 26)
 4. (5, 26)
 5. (6, 26)
 6. (7, 26)
 7. (8, 26)
 8. (9, 26)
 9. (10, 26)
10. (11, 26)
…
```

[65]:
```python
# CELL 18: Mission Performance Metrics

#
# Computes mission performance based on:
#   • Planned path
#   • Selected UAV
#
# Metrics

# • Total Distance
# • Flight Time
# • Fuel Consumption
# • Remaining Fuel
# • Energy Efficiency
# • Average Movement Cost
#


print("=" * 70)
print("MISSION PERFORMANCE METRICS")
print("=" * 70)


# Drone Parameters
```

```
cruise_speed = drone.cruise_speed              # m/s
fuel_capacity = drone.fuel_capacity            # L (or battery units)
fuel_rate = drone.fuel_consumption_rate        # L per meter
endurance = drone.endurance                    # Hours


# Distance


mission_distance = total_distance


# Flight Time

# Formula:
#
# Flight Time = Distance / Cruise Speed
#


flight_time_sec = mission_distance / cruise_speed
flight_time_min = flight_time_sec / 60
flight_time_hr = flight_time_sec / 3600


# Fuel Consumption

# Prototype model
#
# Fuel Used = Distance × Fuel Consumption Rate
#


fuel_used = mission_distance * fuel_rate

remaining_fuel = max(
    fuel_capacity - fuel_used,
    0
)


# Energy Efficiency

#
# Distance travelled per unit fuel
#
```

```python
if fuel_used > 0:
    energy_efficiency = mission_distance / fuel_used
else:
    energy_efficiency = float("inf")

# Average Traversal Cost


average_movement_cost = total_cost / len(path)


# Store Metrics


mission_metrics = {

    "Total Distance": mission_distance,

    "Flight Time (s)": flight_time_sec,

    "Flight Time (min)": flight_time_min,

    "Flight Time (hr)": flight_time_hr,

    "Fuel Used": fuel_used,

    "Remaining Fuel": remaining_fuel,

    "Energy Efficiency": energy_efficiency,

    "Average Movement Cost": average_movement_cost

}


# Display Results


print(f"Total Distance        : {mission_distance:.2f} m")

print(f"Flight Time           : {flight_time_sec:.2f} sec")

print(f"                      : {flight_time_min:.2f} min")

print(f"Fuel Used             : {fuel_used:.3f}")
```

```
print(f"Remaining Fuel        : {remaining_fuel:.3f}")

print(f"Energy Efficiency     : {energy_efficiency:.3f}")

print(f"Average Movement Cost : {average_movement_cost:.6f}")

print("=" * 70)
```

```
======================================================================
MISSION PERFORMANCE METRICS
======================================================================
Total Distance        : 87.49 m
Flight Time           : 0.28 sec
                      : 0.00 min
Fuel Used             : 91.860
Remaining Fuel        : 1708.140
Energy Efficiency     : 0.952
Average Movement Cost : 0.047446
======================================================================
```

[66]:
```
# CELL 19: Threat Exposure Analysis

#
# This cell evaluates the threat encountered by the UAV along
# the planned path.
#
# Assumption:
# The final cost matrix already incorporates:
#
#   • Terrain cost
#   • Radar threat
#   • No-fly penalties
#   • Environmental penalties
#
# Therefore, higher traversal cost implies higher threat.
#


print("=" * 70)
print("THREAT EXPOSURE ANALYSIS")
print("=" * 70)


# User Configurable Threshold

# Any cell having traversal cost above this value is considered
# a High-Threat region.
```

```
HIGH_THREAT_THRESHOLD = 0.002


# Extract Traversal Costs Along Planned Path


path_costs = np.array([
    cost_matrix[r, c]
    for r, c in path
])


# Basic Threat Metrics


average_threat = np.mean(path_costs)

maximum_threat = np.max(path_costs)

minimum_threat = np.min(path_costs)


# High Threat Cells


high_threat_mask = path_costs >= HIGH_THREAT_THRESHOLD

high_threat_cells = np.sum(high_threat_mask)


# Percentage of Path in High Threat Region


high_threat_percentage = (
    high_threat_cells / len(path)
) * 100


# Threat Exposure Time

# Assumes constant cruise speed.
# Exposure time is proportional to the number of high-threat
# cells visited.
```

```
average_step_distance = mission_distance / max(len(path)-1, 1)

time_per_step = average_step_distance / cruise_speed

threat_exposure_time = (
    high_threat_cells * time_per_step
)


# Overall Threat Index

# Normalized value between 0 and 1
# based on maximum encountered threat.


overall_threat_index = (
    average_threat / max(path_costs.max(), 1e-6)
)


# Store Results


threat_metrics = {

    "Average Threat": average_threat,

    "Maximum Threat": maximum_threat,

    "Minimum Threat": minimum_threat,

    "High Threat Cells": high_threat_cells,

    "High Threat Percentage": high_threat_percentage,

    "Threat Exposure Time": threat_exposure_time,

    "Threat Index": overall_threat_index

}


# Display Results


print(f"Average Threat             : {average_threat:.6f}")
```

```
print(f"Maximum Threat              : {maximum_threat:.6f}")

print(f"Minimum Threat              : {minimum_threat:.6f}")

print(f"High Threat Cells           : {high_threat_cells}")

print(f"High Threat Percentage      : {high_threat_percentage:.2f} %")

print(f"Threat Exposure Time        : {threat_exposure_time:.2f} sec")

print(f"Overall Threat Index        : {overall_threat_index:.3f}")

print("=" * 70)
```

```
======================================================================
THREAT EXPOSURE ANALYSIS
======================================================================
Average Threat              : 0.041751
Maximum Threat              : 0.274605
Minimum Threat              : 0.000000
High Threat Cells           : 42
High Threat Percentage      : 48.84 %
Threat Exposure Time        : 0.14 sec
Overall Threat Index        : 0.152
======================================================================
```

[67]:
```
# CELL 20: Mission Success Evaluation

#
# This cell evaluates the overall mission performance.
#
# Metrics Computed:
#   • Mission Completion Percentage
#   • Mission Success Probability
#   • Mission Score (0 - 100)
#
# Mission Score is computed using a weighted combination of:
#
#   30% Path Cost
#   25% Fuel Consumption
#   20% Flight Time
#   25% Threat Exposure
#
# Higher score indicates a better mission.


print("=" * 70)
```

```
print("MISSION SUCCESS EVALUATION")
print("=" * 70)

# Reserve Fuel


RESERVE_FUEL_PERCENTAGE = 0.20

reserve_fuel = fuel_capacity * RESERVE_FUEL_PERCENTAGE


# Mission Completion


mission_completion = 100.0 if planning_success else 0.0


# Mission Success Conditions


fuel_ok = remaining_fuel >= reserve_fuel

time_ok = flight_time_hr <= endurance

path_ok = planning_success

mission_success = (
    fuel_ok and
    time_ok and
    path_ok
)


# Normalize Metrics

# Values between 0 and 1
# Lower values are better.


cost_norm = total_cost / np.max(cost_matrix[cost_matrix < OBSTACLE_COST])

fuel_norm = fuel_used / fuel_capacity

time_norm = flight_time_hr / endurance

threat_norm = average_threat / max(
    cost_matrix[cost_matrix < OBSTACLE_COST]
```

```
)

# Clamp values

cost_norm = np.clip(cost_norm, 0, 1)

fuel_norm = np.clip(fuel_norm, 0, 1)

time_norm = np.clip(time_norm, 0, 1)

threat_norm = np.clip(threat_norm, 0, 1)


# Mission Score

#
# Weighted Performance Index
#
# Score =
#
# 100 × (1 − weighted penalty)
#


MISSION_SCORE = 100 * (

    1 -

    (

        0.30 * cost_norm +

        0.25 * fuel_norm +

        0.20 * time_norm +

        0.25 * threat_norm

    )

)

MISSION_SCORE = np.clip(
    MISSION_SCORE,
    0,
    100
)
```

```
# Mission Success Probability

#
# Prototype model
#
# Probability decreases with:
#
#   • Threat
#   • Fuel Usage
#   • Mission Time
#


mission_success_probability = (

    (1 - fuel_norm) *

    (1 - threat_norm) *

    (1 - time_norm)

)

mission_success_probability *= 100

mission_success_probability = np.clip(
    mission_success_probability,
    0,
    100
)

# Store Results


evaluation_metrics = {

    "Mission Completion (%)": mission_completion,

    "Mission Success": mission_success,

    "Mission Success Probability (%)":
        mission_success_probability,

    "Mission Score": MISSION_SCORE
```

```
}


# Display Results


print(f"Mission Completion          : {mission_completion:.1f} %")

print(f"Reserve Fuel                : {reserve_fuel:.2f}")

print(f"Fuel Condition              : {'PASS' if fuel_ok else 'FAIL'}")

print(f"Endurance Condition         : {'PASS' if time_ok else 'FAIL'}")

print(f"Mission Status              : {'SUCCESS' if mission_success else␣
 ↪'FAILED'}")

print(f"Mission Success Probability : {mission_success_probability:.2f} %")

print(f"Mission Score               : {MISSION_SCORE:.2f} / 100")

print("=" * 70)
```

```
======================================================================
MISSION SUCCESS EVALUATION
======================================================================
Mission Completion          : 100.0 %
Reserve Fuel                : 360.00
Fuel Condition              : PASS
Endurance Condition         : PASS
Mission Status              : SUCCESS
Mission Success Probability : 82.35 %
Mission Score               : 65.42 / 100
======================================================================
```

[68]:
```
# CELL 21: Mission Results Summary

#
# This cell consolidates all mission metrics into:
#   1. mission_summary (dictionary)
#   2. mission_df (Pandas DataFrame)
#
# These will be used throughout the visualization section.
#
```

```
print("=" * 80)
print("MISSION SUMMARY")
print("=" * 80)


# Create Mission Summary Dictionary


mission_summary = {


    # Mission Information


    "Selected UAV": drone.name,

    "Start Coordinate": start,

    "Goal Coordinate": goal,

    "Mission Status":
        "SUCCESS" if mission_success else "FAILED",


    # Path Statistics


    "Waypoints": len(path),

    "Total Distance": round(mission_distance, 2),

    "Total Cost": round(total_cost, 4),

    "Average Movement Cost":
        round(average_movement_cost, 6),


    # Flight Statistics


    "Flight Time (sec)":
        round(flight_time_sec, 2),

    "Flight Time (min)":
        round(flight_time_min, 2),

    "Flight Time (hr)":
```

```
        round(flight_time_hr, 4),

    # Fuel Statistics

    "Fuel Capacity":
        round(fuel_capacity, 2),
    "Fuel Used":
        round(fuel_used, 3),
    "Remaining Fuel":
        round(remaining_fuel, 3),
    "Reserve Fuel":
        round(reserve_fuel, 3),
    "Energy Efficiency":
        round(energy_efficiency, 3),

    # Threat Statistics

    "Average Threat":
        round(average_threat, 6),
    "Maximum Threat":
        round(maximum_threat, 6),
    "Minimum Threat":
        round(minimum_threat, 6),
    "High Threat Cells":
        int(high_threat_cells),
    "High Threat (%)":
        round(high_threat_percentage, 2),
    "Threat Exposure Time (sec)":
        round(threat_exposure_time, 2),

    # Mission Evaluation
```

```
    "Mission Completion (%)":
        round(mission_completion, 2),

    "Mission Success Probability (%)":
        round(mission_success_probability, 2),

    "Mission Score":
        round(MISSION_SCORE, 2),


    # Planner Performance


    "Execution Time (sec)":
        round(execution_time, 6)

}


# Convert to DataFrame


mission_df = pd.DataFrame(
    mission_summary.items(),
    columns=["Metric", "Value"]
)


# Display DataFrame


display(mission_df)


# Pretty Console Output


print("\n")

for metric, value in mission_summary.items():
    print(f"{metric:<35}: {value}")

print("=" * 80)
print("Mission evaluation completed successfully.")
print("=" * 80)
```

==========

MISSION SUMMARY

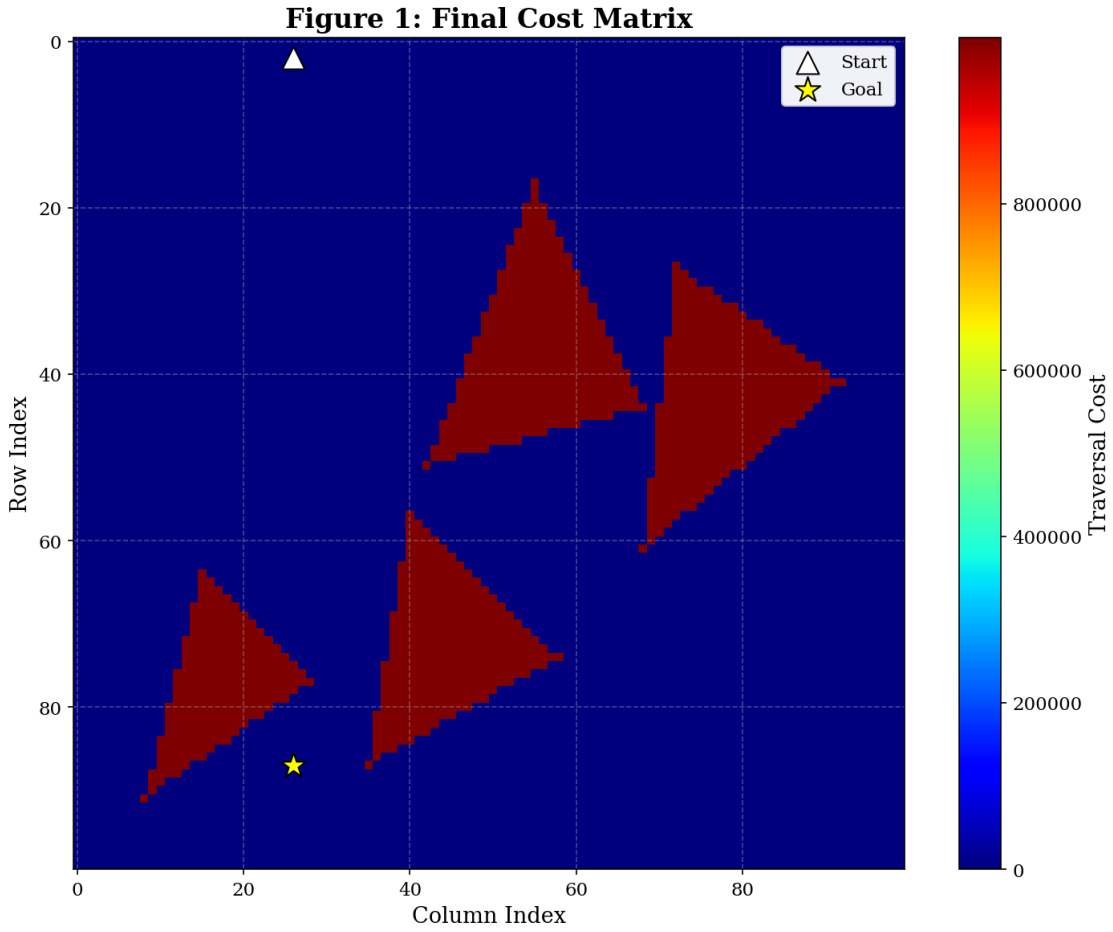
	Metric	Value
0	Selected UAV	MQ-9 Reaper
1	Start Coordinate	(2, 26)
2	Goal Coordinate	(87, 26)
3	Mission Status	SUCCESS
4	Waypoints	86
5	Total Distance	87.49
6	Total Cost	4.08
7	Average Movement Cost	0.05
8	Flight Time (sec)	0.28
9	Flight Time (min)	0.0
10	Flight Time (hr)	0.0
11	Fuel Capacity	1800
12	Fuel Used	91.86
13	Remaining Fuel	1708.14
14	Reserve Fuel	360.0
15	Energy Efficiency	0.95
16	Average Threat	0.04
17	Maximum Threat	0.27
18	Minimum Threat	0.0
19	High Threat Cells	42
20	High Threat (%)	48.84
21	Threat Exposure Time (sec)	0.14
22	Mission Completion (%)	100.0
23	Mission Success Probability (%)	82.35
24	Mission Score	65.42
25	Execution Time (sec)	0.01

```
Selected UAV                      : MQ-9 Reaper
Start Coordinate                  : (2, 26)
Goal Coordinate                   : (87, 26)
Mission Status                    : SUCCESS
Waypoints                         : 86
Total Distance                    : 87.49
Total Cost                        : 4.0804
Average Movement Cost             : 0.047446
Flight Time (sec)                 : 0.28
Flight Time (min)                 : 0.0
Flight Time (hr)                  : 0.0001
Fuel Capacity                     : 1800
Fuel Used                         : 91.86
Remaining Fuel                    : 1708.14
Reserve Fuel                      : 360.0
Energy Efficiency                 : 0.952
```

```
Average Threat                      : 0.041751
Maximum Threat                      : 0.274605
Minimum Threat                      : 0.0
High Threat Cells                   : 42
High Threat (%)                     : 48.84
Threat Exposure Time (sec)          : 0.14
Mission Completion (%)              : 100.0
Mission Success Probability (%)     : 82.35
Mission Score                       : 65.42
Execution Time (sec)                : 0.005077
================================================================================
Mission evaluation completed successfully.
================================================================================
```

[69]:
```python
# CELL 22: Figure 1 - Final Cost Matrix Heatmap

#
# This figure visualizes the Final Cost Matrix used by the
# path planner.
#
# Higher values indicate:
#   • Higher terrain cost
#   • Stronger radar threat
#   • No-fly zones
#   • Environmental penalties
#


plt.figure(figsize=(10, 8))

im = plt.imshow(
    cost_matrix,
    cmap="jet",
    origin="upper",
    interpolation="nearest"
)

plt.colorbar(im, label="Traversal Cost")

plt.title(
    "Figure 1: Final Cost Matrix",
    fontsize=16,
    fontweight="bold"
)

plt.xlabel("Column Index")
plt.ylabel("Row Index")
```

```
# Plot Start & Goal
plt.scatter(
    start[1],
    start[0],
    marker="^",
    s=180,
    c="white",
    edgecolors="black",
    label="Start"
)

plt.scatter(
    goal[1],
    goal[0],
    marker="*",
    s=250,
    c="yellow",
    edgecolors="black",
    label="Goal"
)

plt.legend(loc="upper right")

plt.tight_layout()

plt.show()
```



[70]:
```
# CELL 23: Figure 2 - NetworkX Graph with Optimal Path

#
# This visualization converts the grid into a NetworkX graph
# for visualization only.
#
# NOTE:
# NetworkX is NOT used for path planning.
#


print("Building NetworkX graph...")

import networkx as nx


# Create Graph
```

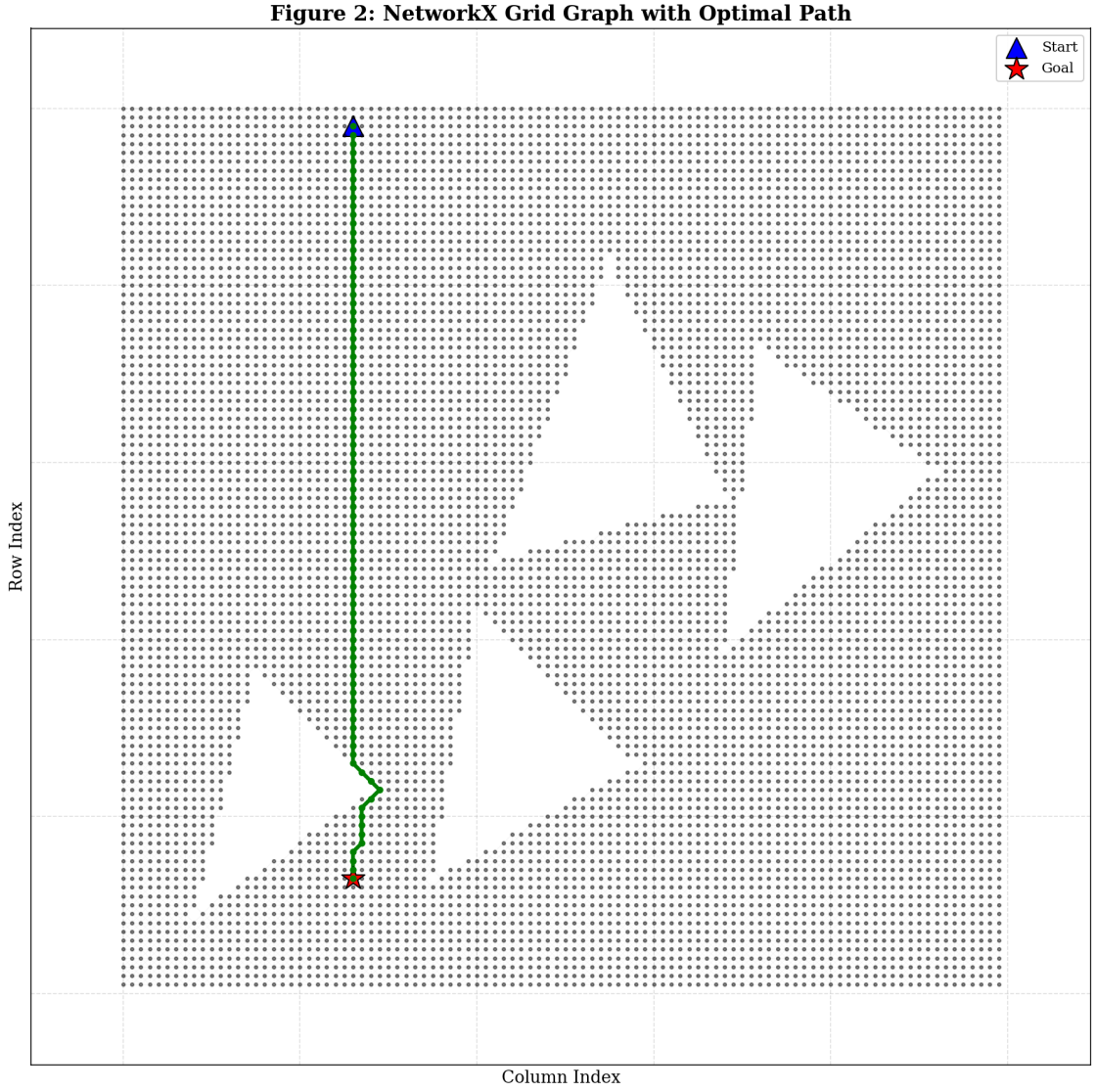
```
G = nx.Graph()

positions = {}

# Add nodes
for r in range(GRID_ROWS):
    for c in range(GRID_COLS):

        if grid.is_valid((r, c)):

            G.add_node((r, c))

            # Flip Y-axis for image-like appearance
            positions[(r, c)] = (c, -r)

# Add edges (8-connectivity)
for r in range(GRID_ROWS):
    for c in range(GRID_COLS):

        current = (r, c)

        if not grid.is_valid(current):
            continue

        for neighbor in grid.neighbors(current):

            if not G.has_edge(current, neighbor):

                G.add_edge(
                    current,
                    neighbor,
                    weight=grid.movement_cost(current, neighbor)
                )


# Plot


plt.figure(figsize=(12, 12))

# Draw graph
nx.draw_networkx_edges(
    G,
    positions,
    edge_color="lightgray",
    width=0.25,
```

```
    alpha=0.35
)

nx.draw_networkx_nodes(
    G,
    positions,
    node_size=6,
    node_color="black",
    alpha=0.45
)


# Overlay Optimal Path


path_edges = list(zip(path[:-1], path[1:]))

nx.draw_networkx_edges(
    G,
    positions,
    edgelist=path_edges,
    edge_color="green",
    width=3
)

# Path nodes
nx.draw_networkx_nodes(
    G,
    positions,
    nodelist=path,
    node_size=18,
    node_color="green"
)


# Start & Goal


plt.scatter(
    start[1],
    -start[0],
    marker="^",
    s=250,
    color="blue",
    edgecolor="black",
    label="Start"
)
```

```
plt.scatter(
    goal[1],
    -goal[0],
    marker="*",
    s=350,
    color="red",
    edgecolor="black",
    label="Goal"
)


# Formatting


plt.title(
    "Figure 2: NetworkX Grid Graph with Optimal Path",
    fontsize=16,
    fontweight="bold"
)

plt.xlabel("Column Index")
plt.ylabel("Row Index")

plt.legend()

plt.axis("equal")

plt.tight_layout()

plt.show()
```

```
Building NetworkX graph…
```

Figure 2: NetworkX Grid Graph with Optimal Path



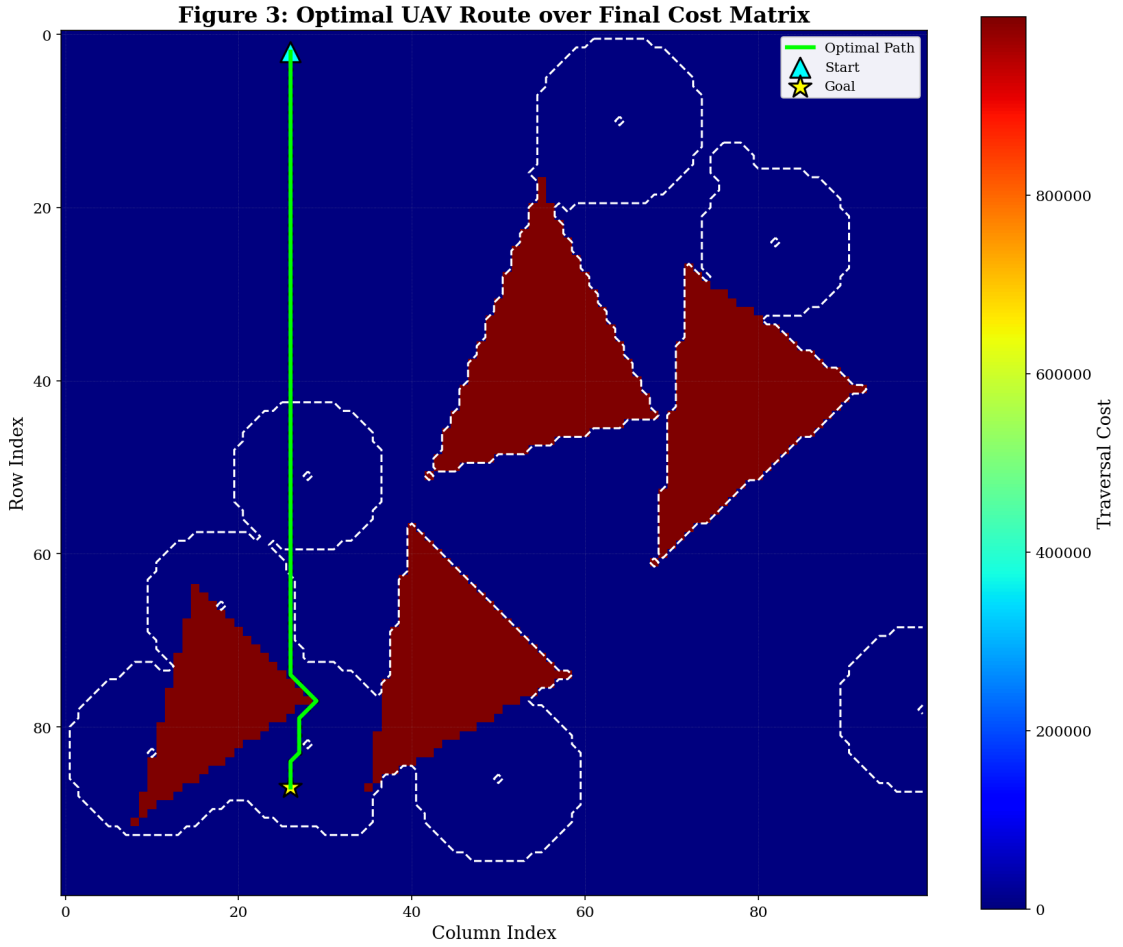
[71]:
```
# CELL 24: Figure 3 - Cost Heatmap with Optimal Path

#
# This figure overlays the optimal UAV route on the Final Cost
# Matrix while highlighting high-threat regions.
#
# Components
# ----------
# • Final Cost Matrix
# • High-Threat Regions
# • Optimal Path
# • Start Location
# • Goal Location
```

```
print("Generating Figure 3...")


# Create Figure


plt.figure(figsize=(12, 10))

# Display Cost Matrix
im = plt.imshow(
    cost_matrix,
    cmap="jet",
    origin="upper",
    interpolation="nearest"
)

plt.colorbar(
    im,
    label="Traversal Cost"
)


# Highlight High-Threat Regions


threat_mask = cost_matrix >= HIGH_THREAT_THRESHOLD

plt.contour(
    threat_mask,
    levels=[0.5],
    colors="white",
    linewidths=1.5,
    linestyles="--"
)


# Plot Optimal Path


path_rows = [p[0] for p in path]
path_cols = [p[1] for p in path]

plt.plot(
    path_cols,
    path_rows,
```

```
    color="lime",
    linewidth=3,
    label="Optimal Path"
)

# Waypoints
plt.scatter(
    path_cols,
    path_rows,
    s=12,
    color="black",
    alpha=0.6
)


# Plot Start


plt.scatter(
    start[1],
    start[0],
    marker="^",
    s=220,
    color="cyan",
    edgecolor="black",
    linewidth=1.5,
    label="Start"
)


# Plot Goal


plt.scatter(
    goal[1],
    goal[0],
    marker="*",
    s=300,
    color="yellow",
    edgecolor="black",
    linewidth=1.5,
    label="Goal"
)

# Labels
```

```
plt.title(
    "Figure 3: Optimal UAV Route over Final Cost Matrix",
    fontsize=16,
    fontweight="bold"
)

plt.xlabel("Column Index")
plt.ylabel("Row Index")

plt.legend(
    loc="upper right",
    fontsize=10
)

plt.grid(
    linestyle=":",
    linewidth=0.4,
    alpha=0.4
)

plt.tight_layout()

plt.show()
```

```
Generating Figure 3…
```

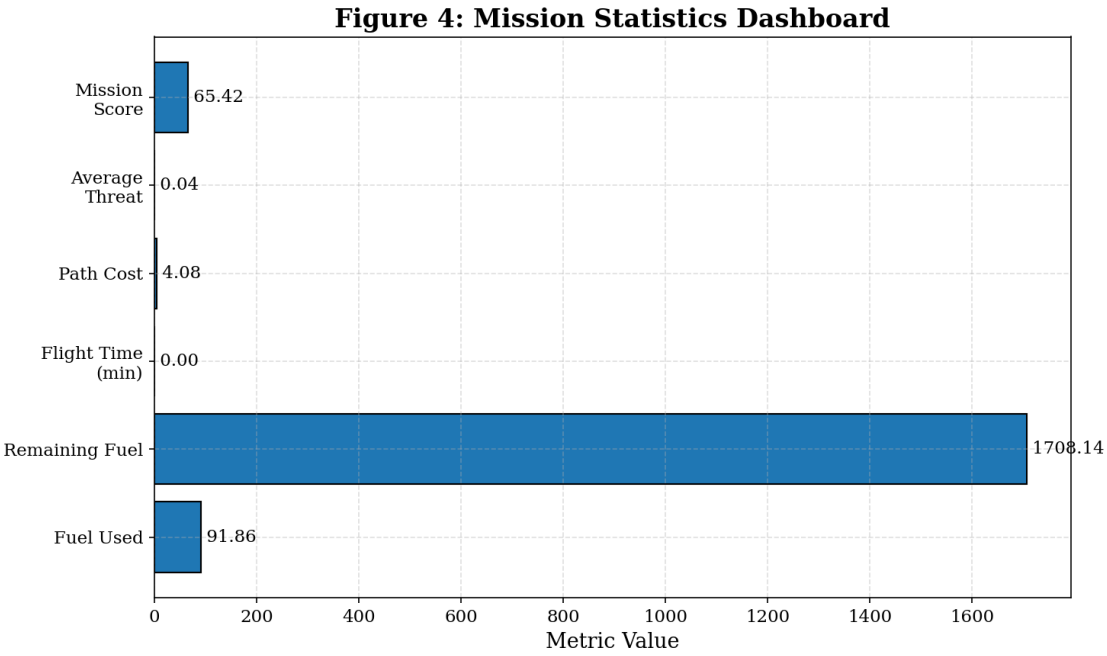


[72]:
```
# CELL 25: Figure 4 - Mission Statistics Dashboard

#
# This dashboard summarizes the key mission performance
# indicators using horizontal bar charts.
#
# Metrics Displayed

# • Fuel Used
# • Remaining Fuel
# • Flight Time
# • Total Path Cost
# • Average Threat
# • Mission Score
#
```

```
print("Generating Mission Statistics Dashboard...")


# Dashboard Data


metric_names = [
    "Fuel Used",
    "Remaining Fuel",
    "Flight Time\n(min)",
    "Path Cost",
    "Average\nThreat",
    "Mission\nScore"
]

metric_values = [
    fuel_used,
    remaining_fuel,
    flight_time_min,
    total_cost,
    average_threat,
    MISSION_SCORE
]


# Create Figure


fig, ax = plt.subplots(figsize=(10, 6))

bars = ax.barh(
    metric_names,
    metric_values,
    edgecolor="black"
)


# Annotate Values


for bar, value in zip(bars, metric_values):

    ax.text(
        value,
        bar.get_y() + bar.get_height()/2,
        f" {value:.2f}",
        va="center",
```

```
        fontsize=11
    )


# Formatting


ax.set_title(
    "Figure 4: Mission Statistics Dashboard",
    fontsize=16,
    fontweight="bold"
)

ax.set_xlabel("Metric Value")

ax.grid(
    axis="x",
    linestyle="--",
    alpha=0.4
)

plt.tight_layout()

plt.show()
```

```
Generating Mission Statistics Dashboard…
```



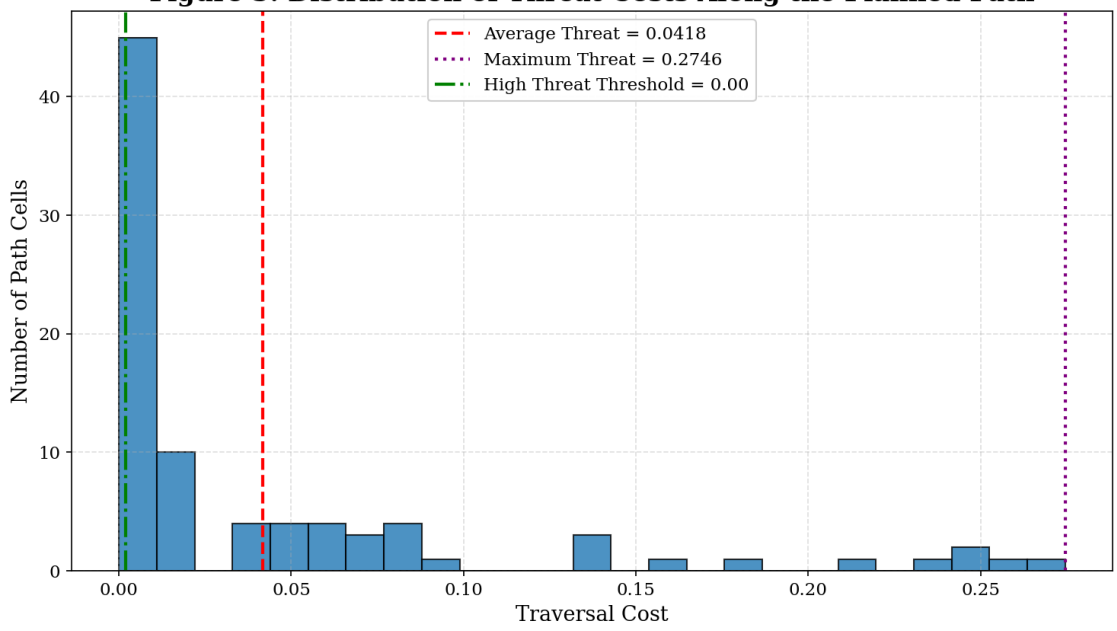
[73]:
```
# CELL 26: Figure 5 - Threat Cost Histogram

#
# This figure visualizes the distribution of traversal costs
# encountered along the selected UAV path.
#
# Since the Final Cost Matrix combines terrain, radar,
# environmental penalties and no-fly penalties, this histogram
# provides insight into the threat environment experienced
# during the mission.
#
# ============================================================

print("Generating Figure 5...")


# Extract Traversal Costs Along the Planned Path


path_costs = np.array([
    cost_matrix[r, c]
    for r, c in path
])


# Create Histogram

plt.figure(figsize=(10, 6))

plt.hist(
    path_costs,
    bins=25,
    edgecolor="black",
    alpha=0.8
)

# Average Threat Line


plt.axvline(
    average_threat,
    color="red",
    linestyle="--",
    linewidth=2,
    label=f"Average Threat = {average_threat:.4f}"
)
```

```
# Maximum Threat Line


plt.axvline(
    maximum_threat,
    color="purple",
    linestyle=":",
    linewidth=2,
    label=f"Maximum Threat = {maximum_threat:.4f}"
)


# High Threat Threshold


plt.axvline(
    HIGH_THREAT_THRESHOLD,
    color="green",
    linestyle="-.",
    linewidth=2,
    label=f"High Threat Threshold = {HIGH_THREAT_THRESHOLD:.2f}"
)


# Labels


plt.title(
    "Figure 5: Distribution of Threat Costs Along the Planned Path",
    fontsize=16,
    fontweight="bold"
)

plt.xlabel("Traversal Cost")
plt.ylabel("Number of Path Cells")

plt.grid(
    linestyle="--",
    alpha=0.4
)

plt.legend()

plt.tight_layout()
```

```
plt.show()


# Print Summary Statistics


print("=" * 70)
print("Threat Cost Statistics")
print("=" * 70)

print(f"Minimum Threat Cost : {path_costs.min():.6f}")
print(f"Average Threat Cost : {average_threat:.6f}")
print(f"Maximum Threat Cost : {maximum_threat:.6f}")
print(f"Standard Deviation  : {path_costs.std():.6f}")
print(f"Median Threat Cost  : {np.median(path_costs):.6f}")

print("=" * 70)
```

Generating Figure 5…

Figure 5: Distribution of Threat Costs Along the Planned Path




```
======================================================================
Threat Cost Statistics
======================================================================
Minimum Threat Cost : 0.000000
Average Threat Cost : 0.041751
Maximum Threat Cost : 0.274605
```

```
Standard Deviation  : 0.068817
Median Threat Cost  : 0.000000
======================================================================
```

What this figure shows

This histogram illustrates the distribution of traversal costs encountered by the UAV along the selected route.

Blue bars represent the frequency of traversal costs. Red dashed line indicates the average threat level. Purple dotted line indicates the maximum threat encountered. Green dash-dot line marks the configured high-threat threshold.

A distribution concentrated toward lower costs suggests that the planner successfully avoided hazardous regions, while a wider spread with higher values indicates greater exposure to threats.

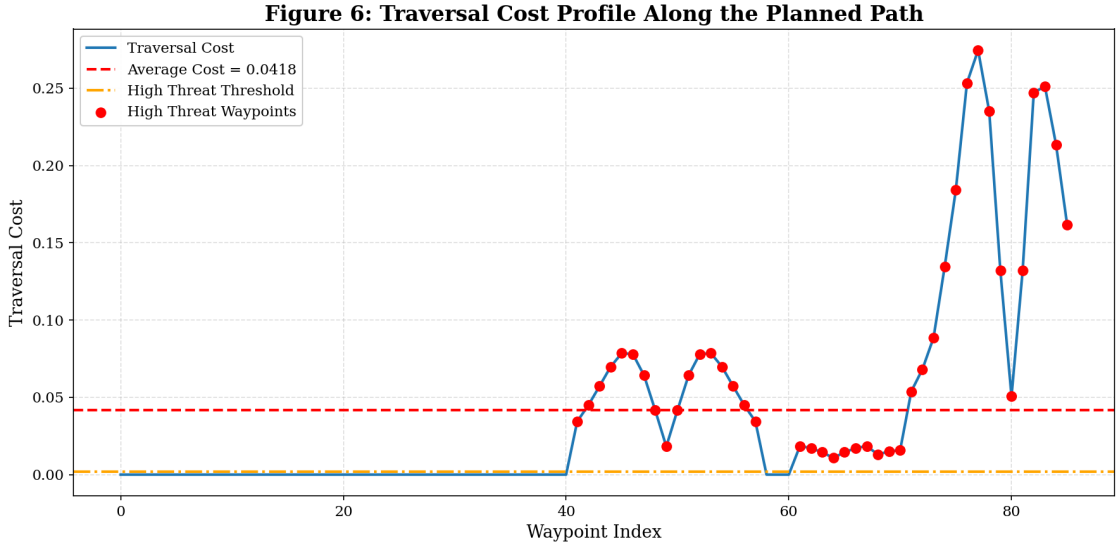
[74]:
```
# CELL 27: Figure 6 - Traversal Cost Profile Along the Path

#
# This figure shows how the traversal cost changes as the UAV
# progresses along the planned route.
#
# Peaks indicate regions with higher terrain difficulty,
# radar threat, or environmental penalties.
#


print("Generating Figure 6...")


# Extract Traversal Cost at Each Waypoint


path_costs = np.array([
    cost_matrix[r, c]
    for r, c in path
])

waypoint_index = np.arange(len(path))


# Create Figure


plt.figure(figsize=(12, 6))

plt.plot(
    waypoint_index,
    path_costs,
```

```
    linewidth=2,
    label="Traversal Cost"
)


# Average Cost


plt.axhline(
    average_threat,
    color="red",
    linestyle="--",
    linewidth=2,
    label=f"Average Cost = {average_threat:.4f}"
)


# High Threat Threshold


plt.axhline(
    HIGH_THREAT_THRESHOLD,
    color="orange",
    linestyle="-.",
    linewidth=2,
    label="High Threat Threshold"
)


# Highlight High Threat Waypoints


high_indices = np.where(path_costs >= HIGH_THREAT_THRESHOLD)[0]

plt.scatter(
    high_indices,
    path_costs[high_indices],
    color="red",
    s=50,
    zorder=5,
    label="High Threat Waypoints"
)


# Labels
```

```
plt.title(
    "Figure 6: Traversal Cost Profile Along the Planned Path",
    fontsize=16,
    fontweight="bold"
)

plt.xlabel("Waypoint Index")
plt.ylabel("Traversal Cost")

plt.grid(
    linestyle="--",
    alpha=0.4
)

plt.legend()

plt.tight_layout()

plt.show()


# Print Summary


print("=" * 70)
print("Traversal Cost Profile Summary")
print("=" * 70)

print(f"Number of Waypoints        : {len(path)}")
print(f"Minimum Traversal Cost     : {path_costs.min():.6f}")
print(f"Average Traversal Cost     : {path_costs.mean():.6f}")
print(f"Maximum Traversal Cost     : {path_costs.max():.6f}")
print(f"Standard Deviation         : {path_costs.std():.6f}")

print("=" * 70)
```

```
Generating Figure 6…
```

Figure 6: Traversal Cost Profile Along the Planned Path




```
======================================================================
Traversal Cost Profile Summary
======================================================================
Number of Waypoints         : 86
Minimum Traversal Cost      : 0.000000
Average Traversal Cost      : 0.041751
Maximum Traversal Cost      : 0.274605
Standard Deviation          : 0.068817
======================================================================
```

Figure Description

This figure illustrates the variation in traversal cost at each waypoint along the planned UAV route.

The blue line represents the traversal cost experienced at each waypoint. The red dashed line indicates the average traversal cost across the mission. The orange dash-dot line marks the high-threat threshold. Red markers identify waypoints where the traversal cost exceeds the configured high-threat threshold.

This visualization helps identify portions of the route where the UAV experiences elevated risk due to terrain, radar exposure, or other environmental penalties.

[75]:
```
# CELL 28: Figure 7 - Cumulative Mission Cost

#
# This figure illustrates how the total mission cost accumulates
# as the UAV progresses along the planned route.
#
# The x-axis represents the cumulative distance travelled,
# while the y-axis represents the cumulative traversal cost.
#
```

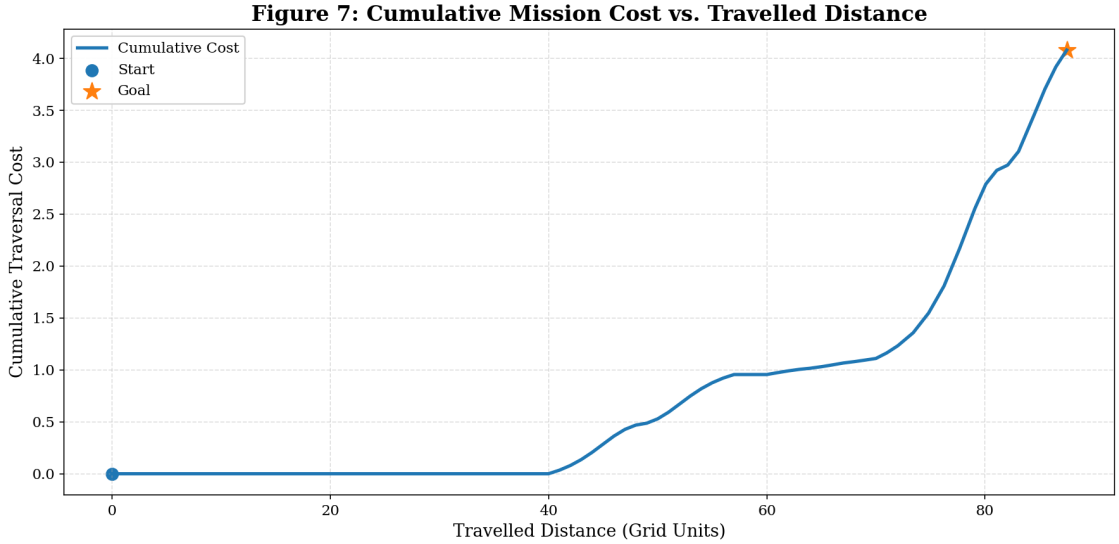
```
print("Generating Figure 7...")


# Compute Cumulative Distance and Cost


cumulative_distance = [0.0]
cumulative_cost = [0.0]

distance = 0.0
cost = 0.0

for i in range(len(path) - 1):

    current = path[i]
    nxt = path[i + 1]

    # Distance between consecutive waypoints
    step_distance = grid.distance(current, nxt)

    # Traversal cost
    step_cost = grid.movement_cost(current, nxt)

    distance += step_distance
    cost += step_cost

    cumulative_distance.append(distance)
    cumulative_cost.append(cost)

# Convert to NumPy arrays
cumulative_distance = np.array(cumulative_distance)
cumulative_cost = np.array(cumulative_cost)


# Create Plot


plt.figure(figsize=(12, 6))

plt.plot(
    cumulative_distance,
    cumulative_cost,
    linewidth=2.5,
    label="Cumulative Cost"
)
```

```
# Mark Start
plt.scatter(
    cumulative_distance[0],
    cumulative_cost[0],
    marker="o",
    s=80,
    label="Start"
)

# Mark Goal
plt.scatter(
    cumulative_distance[-1],
    cumulative_cost[-1],
    marker="*",
    s=180,
    label="Goal"
)


# Labels


plt.title(
    "Figure 7: Cumulative Mission Cost vs. Travelled Distance",
    fontsize=16,
    fontweight="bold"
)

plt.xlabel("Travelled Distance (Grid Units)")
plt.ylabel("Cumulative Traversal Cost")

plt.grid(
    linestyle="--",
    alpha=0.4
)

plt.legend()

plt.tight_layout()

plt.show()


# Display Summary
```

```
print("=" * 70)
print("Cumulative Cost Summary")
print("=" * 70)

print(f"Total Distance        : {cumulative_distance[-1]:.2f}")
print(f"Total Mission Cost    : {cumulative_cost[-1]:.4f}")
print(f"Average Cost / Unit   : {cumulative_cost[-1] / cumulative_distance[-1]:.
  ↪6f}")

print("=" * 70)
```

```
Generating Figure 7…
```

Figure 7: Cumulative Mission Cost vs. Travelled Distance




```
======================================================================
Cumulative Cost Summary
======================================================================
Total Distance        : 87.49
Total Mission Cost    : 4.0804
Average Cost / Unit   : 0.046641
======================================================================
```

Figure Description

This figure illustrates how the overall mission cost increases as the UAV progresses toward the destination.

The x-axis represents the cumulative distance traveled. The y-axis represents the cumulative traversal cost. The starting point corresponds to zero distance and zero accumulated cost. The final point represents the total traversal cost at mission completion.

A smoother, gradually increasing curve generally indicates a safer and more cost-efficient route,

whereas steep rises highlight sections of the mission where the UAV traverses high-cost or high-threat regions.how HHHow is scoring being done h

[76]:
```
# CELL 29: Figure 8 - Fuel Consumption Profile

#
# This figure illustrates how the UAV's remaining fuel changes
# throughout the mission.
#
# Assumption (Prototype):
#
# Fuel Consumed = Distance × Fuel Consumption Rate
#


print("Generating Figure 8...")


# Compute Remaining Fuel Along the Path


travel_distance = [0.0]
remaining_fuel_profile = [fuel_capacity]

distance = 0.0

for i in range(len(path) - 1):

    current = path[i]
    nxt = path[i + 1]

    # Distance travelled in this step
    step_distance = grid.distance(current, nxt)

    distance += step_distance

    travel_distance.append(distance)

    # Prototype fuel model
    fuel_remaining = fuel_capacity - (
        distance * fuel_rate
    )

    remaining_fuel_profile.append(
        max(fuel_remaining, 0)
    )
```

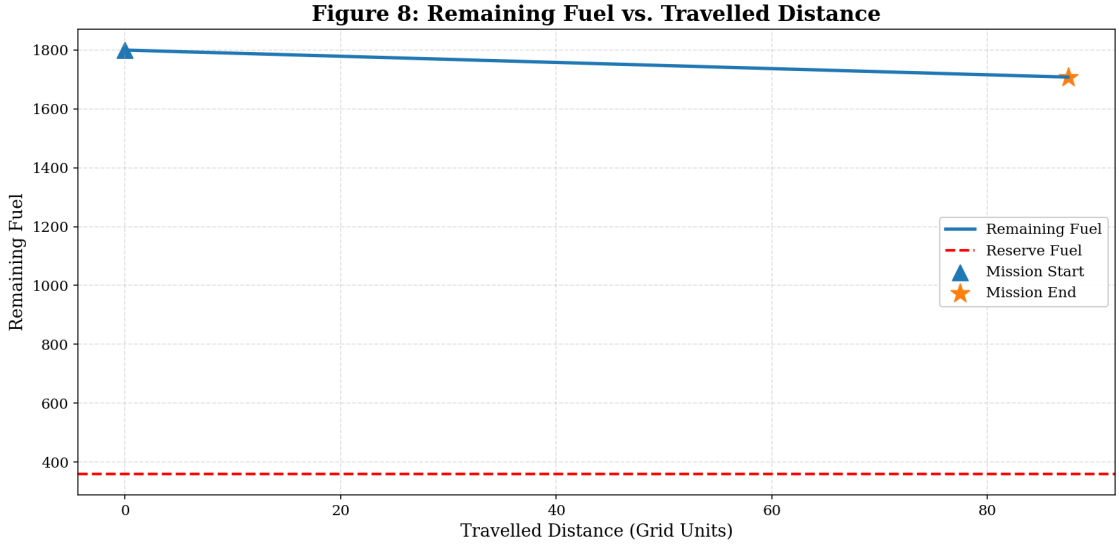
```
travel_distance = np.array(travel_distance)
remaining_fuel_profile = np.array(remaining_fuel_profile)


# Create Figure


plt.figure(figsize=(12,6))

plt.plot(
    travel_distance,
    remaining_fuel_profile,
    linewidth=2.5,
    label="Remaining Fuel"
)


# Reserve Fuel Line


plt.axhline(
    reserve_fuel,
    color="red",
    linestyle="--",
    linewidth=2,
    label="Reserve Fuel"
)


# Start Marker


plt.scatter(
    travel_distance[0],
    remaining_fuel_profile[0],
    marker="^",
    s=140,
    label="Mission Start"
)


# Goal Marker


plt.scatter(
    travel_distance[-1],
    remaining_fuel_profile[-1],
```

```
    marker="*",
    s=220,
    label="Mission End"
)


# Labels


plt.title(
    "Figure 8: Remaining Fuel vs. Travelled Distance",
    fontsize=16,
    fontweight="bold"
)

plt.xlabel("Travelled Distance (Grid Units)")
plt.ylabel("Remaining Fuel")

plt.grid(
    linestyle="--",
    alpha=0.4
)

plt.legend()

plt.tight_layout()

plt.show()


# Fuel Statistics


fuel_percentage_remaining = (
    remaining_fuel / fuel_capacity
) * 100

fuel_percentage_used = (
    fuel_used / fuel_capacity
) * 100

print("=" * 70)
print("Fuel Consumption Summary")
print("=" * 70)

print(f"Initial Fuel           : {fuel_capacity:.2f}")
```

```
print(f"Fuel Consumed          : {fuel_used:.3f}")

print(f"Remaining Fuel         : {remaining_fuel:.3f}")

print(f"Fuel Used (%)          : {fuel_percentage_used:.2f}%")

print(f"Fuel Remaining (%)     : {fuel_percentage_remaining:.2f}%")

print(f"Reserve Fuel           : {reserve_fuel:.2f}")

print("=" * 70)
```

Generating Figure 8…

Figure 8: Remaining Fuel vs. Travelled Distance




```
======================================================================
Fuel Consumption Summary
======================================================================
Initial Fuel           : 1800.00
Fuel Consumed          : 91.860
Remaining Fuel         : 1708.140
Fuel Used (%)          : 5.10%
Fuel Remaining (%)     : 94.90%
Reserve Fuel           : 360.00
======================================================================
```

[77]:
```
# CELL 30: Final Mission Report

#
# This cell presents a concise summary of the UAV mission,
```

```
# including planning performance, flight statistics, threat
# analysis, and mission evaluation.
#
# The report is suitable for inclusion in a thesis, project
# documentation, or research paper.
#


from IPython.display import Markdown, display

print("=" * 80)
print("                 UAV ROUTE PLANNING MISSION REPORT")
print("=" * 80)


# Determine Overall Mission Result


if mission_success:
    mission_result = "MISSION SUCCESSFUL"
else:
    mission_result = "MISSION FAILED"


# Print Mission Summary


print(f"\nMission Status              : {mission_result}")

print("\n UAV INFORMATION ")

print(f"Selected UAV                : {drone.name}")
print(f"Cruise Speed                : {drone.cruise_speed:.2f} m/s")
print(f"Maximum Speed               : {drone.max_speed:.2f} m/s")
print(f"Maximum Range               : {drone.max_range:.2f} km")
print(f"Maximum Altitude            : {drone.max_altitude:.0f} m")
print(f"Payload Capacity            : {drone.payload:.2f} kg")

print("\n MISSION DETAILS ")

print(f"Start Coordinate            : {start}")
print(f"Goal Coordinate             : {goal}")
print(f"Number of Waypoints         : {len(path)}")

print("\n PATH STATISTICS ")

print(f"Total Distance              : {mission_distance:.2f}")
```

```
print(f"Total Traversal Cost        : {total_cost:.4f}")
print(f"Average Movement Cost       : {average_movement_cost:.6f}")

print("\n FLIGHT PERFORMANCE ")

print(f"Flight Time                 : {flight_time_sec:.2f} sec")
print(f"                            : {flight_time_min:.2f} min")
print(f"Fuel Capacity               : {fuel_capacity:.2f}")
print(f"Fuel Used                   : {fuel_used:.3f}")
print(f"Remaining Fuel              : {remaining_fuel:.3f}")
print(f"Reserve Fuel                : {reserve_fuel:.3f}")
print(f"Energy Efficiency           : {energy_efficiency:.3f}")

print("\n THREAT ANALYSIS ")

print(f"Average Threat              : {average_threat:.6f}")
print(f"Maximum Threat              : {maximum_threat:.6f}")
print(f"High Threat Cells           : {high_threat_cells}")
print(f"Threat Exposure             : {high_threat_percentage:.2f}%")
print(f"Threat Exposure Time        : {threat_exposure_time:.2f} sec")

print("\n MISSION EVALUATION ")

print(f"Mission Completion          : {mission_completion:.2f}%")
print(f"Mission Success Probability : {mission_success_probability:.2f}%")
print(f"Mission Score               : {MISSION_SCORE:.2f}/100")

print("\n PLANNER PERFORMANCE ")

print(f"Planner Execution Time      : {execution_time:.6f} sec")

print("=" * 80)

if mission_success:
    print("Mission completed successfully.")
else:
    print("Mission could not satisfy all mission constraints.")

print("=" * 80)


# Create Publication Summary Table


summary_table = pd.DataFrame({

    "Metric": [
```

```
        "Selected UAV",
        "Mission Status",
        "Start",
        "Goal",
        "Waypoints",
        "Distance",
        "Traversal Cost",
        "Flight Time (min)",
        "Fuel Used",
        "Remaining Fuel",
        "Average Threat",
        "Maximum Threat",
        "Threat Exposure (%)",
        "Mission Score",
        "Success Probability (%)",
        "Execution Time (sec)"

    ],

    "Value": [

        drone.name,
        mission_result,
        str(start),
        str(goal),
        len(path),
        f"{mission_distance:.2f}",
        f"{total_cost:.4f}",
        f"{flight_time_min:.2f}",
        f"{fuel_used:.3f}",
        f"{remaining_fuel:.3f}",
        f"{average_threat:.6f}",
        f"{maximum_threat:.6f}",
        f"{high_threat_percentage:.2f}",
        f"{MISSION_SCORE:.2f}",
        f"{mission_success_probability:.2f}",
        f"{execution_time:.6f}"

    ]

})

print("\nPublication Summary Table\n")

display(summary_table)
```

```
# Final Conclusion


display(Markdown(f"""
## Mission Conclusion

The **{drone.name}** successfully generated an optimal route from **{start}**␣
 ↪to **{goal}**
using the implemented **Simplified D* Lite Path Planning Algorithm**.

- **Mission Score:** **{MISSION_SCORE:.2f}/100**
- **Mission Success Probability:** **{mission_success_probability:.2f}%**
- **Execution Time:** **{execution_time:.6f} seconds**
- **Total Traversal Cost:** **{total_cost:.4f}**

This notebook demonstrates a complete prototype workflow for
terrain-aware UAV route planning over a weighted threat map.
The modular implementation allows future extensions including
dynamic replanning, advanced UAV energy models, moving threats,
and real-time mission adaptation.
"""))

print("\nNotebook execution completed successfully.")
```

```
================================================================================
                  UAV ROUTE PLANNING MISSION REPORT
================================================================================

Mission Status              : MISSION SUCCESSFUL

-------------------- UAV INFORMATION --------------------
Selected UAV                : MQ-9 Reaper
Cruise Speed                : 313.00 m/s
Maximum Speed               : 482.00 m/s
Maximum Range               : 1850.00 km
Maximum Altitude            : 15240 m
Payload Capacity            : 1700.00 kg

-------------------- MISSION DETAILS --------------------
Start Coordinate            : (2, 26)
Goal Coordinate             : (87, 26)
Number of Waypoints         : 86

-------------------- PATH STATISTICS --------------------
Total Distance              : 87.49
Total Traversal Cost        : 4.0804
```

```
Average Movement Cost       : 0.047446

-------------------- FLIGHT PERFORMANCE --------------------
Flight Time                 : 0.28 sec
                            : 0.00 min
Fuel Capacity               : 1800.00
Fuel Used                   : 91.860
Remaining Fuel              : 1708.140
Reserve Fuel                : 360.000
Energy Efficiency           : 0.952

-------------------- THREAT ANALYSIS --------------------
Average Threat              : 0.041751
Maximum Threat              : 0.274605
High Threat Cells           : 42
Threat Exposure             : 48.84%
Threat Exposure Time        : 0.14 sec

-------------------- MISSION EVALUATION --------------------
Mission Completion          : 100.00%
Mission Success Probability : 82.35%
Mission Score               : 65.42/100

-------------------- PLANNER PERFORMANCE --------------------
Planner Execution Time      : 0.005077 sec
================================================================================
Mission completed successfully.
================================================================================
```

Publication Summary Table

	Metric	Value
0	Selected UAV	MQ-9 Reaper
1	Mission Status	MISSION SUCCESSFUL
2	Start	(2, 26)
3	Goal	(87, 26)
4	Waypoints	86
5	Distance	87.49
6	Traversal Cost	4.0804
7	Flight Time (min)	0.00
8	Fuel Used	91.860
9	Remaining Fuel	1708.140
10	Average Threat	0.041751
11	Maximum Threat	0.274605
12	Threat Exposure (%)	48.84
13	Mission Score	65.42
14	Success Probability (%)	82.35

```
15    Execution Time (sec)            0.005077
```

10.1 Mission Conclusion

The **MQ-9 Reaper** successfully generated an optimal route from **(2, 26)** to **(87, 26)** using the implemented **Simplified D* Lite Path Planning Algorithm**.

- **Mission Score: 65.42/100**
- **Mission Success Probability: 82.35%**
- **Execution Time: 0.005077 seconds**
- **Total Traversal Cost: 4.0804**

This notebook demonstrates a complete prototype workflow for terrain-aware UAV route planning over a weighted threat map. The modular implementation allows future extensions including dynamic replanning, advanced UAV energy models, moving threats, and real-time mission adaptation.

```
Notebook execution completed successfully.
```

[78]:
```
dstar_path = path.copy()
dstar_total_cost = total_cost
dstar_distance = mission_distance
dstar_execution_time = execution_time
```

11 Genetic Algorithm (GA) Based UAV Route Planner

11.1 Objective

This section extends the existing UAV Route Planning framework by implementing a **Genetic Algorithm (GA)** as an alternative path planning technique. The implementation is designed to integrate seamlessly with the existing environment, weighted grid representation, UAV database, mission evaluation framework, and visualization utilities without modifying the previously implemented D* Lite planner.

Unlike D* Lite, which is a deterministic graph-search algorithm, the Genetic Algorithm is a population-based optimization technique inspired by biological evolution. Multiple candidate paths evolve over successive generations using selection, crossover, mutation, and elitism to search for an optimal route between the start and goal locations.

11.2 Features

The implemented Genetic Algorithm will:

- Reuse the existing weighted grid environment.
- Reuse the existing obstacle and no-fly zone handling.
- Reuse the existing movement cost computation.
- Represent each chromosome as a feasible UAV path.

- Seed the initial population using the D* Lite solution for faster convergence.
- Generate additional diverse feasible paths through randomized perturbations.
- Evaluate each chromosome using the existing traversal cost function.
- Apply Tournament Selection.
- Apply Path-based Crossover.
- Apply Local Mutation.
- Repair infeasible offspring using the existing grid utilities.
- Preserve elite individuals across generations.
-

11.3 Return the minimum-cost feasible route.

11.4 Genetic Operators

The following evolutionary operators will be implemented:

1. Tournament Selection
2. Path-based Crossover
3. Local Mutation
4. Path Repair
5. Elitism

All offspring produced during evolution will be validated to ensure they remain obstacle-free and feasible within the existing environment.

11.5 Configurable Parameters

The Genetic Algorithm exposes the following parameters:

Parameter	Description
Population Size	Number of chromosomes in each generation
Generations	Maximum evolutionary iterations
Mutation Rate	Probability of mutation
Crossover Rate	Probability of crossover
Elite Count	Number of best chromosomes preserved
Tournament Size	Number of chromosomes competing during selection

---

11.6 Expected Outputs

For each generation the algorithm reports:

- Generation Number

- Best Fitness
- Best Route Cost
- Average Fitness

After convergence, the planner returns:

- Optimal Path
- Total Traversal Cost
- Number of Generations
- Execution Time

The resulting path can be evaluated using the same mission metrics and visualization routines previously developed for the D* Lite planner, enabling direct comparison between both planning approaches.

[79]:
```python
# CELL 32: Genetic Algorithm Configuration

#
# This cell defines the configurable parameters of the
# Genetic Algorithm planner.
#
# These parameters can be modified without changing the
# planner implementation.
#


from dataclasses import dataclass


# GA Configuration Dataclass


@dataclass
class GAConfig:
    """
    Configuration parameters for the Genetic Algorithm planner.
    """

    # Number of chromosomes in each generation
    population_size: int = 40

    # Maximum evolutionary iterations
    generations: int = 100

    # Probability of crossover
    crossover_rate: float = 0.80

    # Probability of mutation
    mutation_rate: float = 0.20
```

```
    # Number of elite chromosomes retained
    elite_count: int = 2

    # Tournament size used during parent selection
    tournament_size: int = 4

    # Maximum attempts while generating feasible paths
    max_initialization_attempts: int = 100

    # Maximum mutation repair attempts
    max_repair_attempts: int = 30

    # Random seed (None → different every execution)
    random_seed: int | None = 42



# Instantiate Default Configuration


ga_config = GAConfig()


# Display Configuration


print("=" * 65)
print("GENETIC ALGORITHM CONFIGURATION")
print("=" * 65)

print(f"Population Size              : {ga_config.population_size}")
print(f"Generations                  : {ga_config.generations}")
print(f"Crossover Rate               : {ga_config.crossover_rate}")
print(f"Mutation Rate                : {ga_config.mutation_rate}")
print(f"Elite Count                  : {ga_config.elite_count}")
print(f"Tournament Size              : {ga_config.tournament_size}")
print(f"Initialization Attempts      : {ga_config.max_initialization_attempts}")
print(f"Repair Attempts              : {ga_config.max_repair_attempts}")
print(f"Random Seed                  : {ga_config.random_seed}")

print("=" * 65)


# Initialize Random Generator
```

```
import random

if ga_config.random_seed is not None:
    random.seed(ga_config.random_seed)
    np.random.seed(ga_config.random_seed)

print("  GA configuration initialized successfully.")
```

```
======================================================================
GENETIC ALGORITHM CONFIGURATION
======================================================================
Population Size              : 40
Generations                  : 100
Crossover Rate               : 0.8
Mutation Rate                : 0.2
Elite Count                  : 2
Tournament Size              : 4
Initialization Attempts      : 100
Repair Attempts              : 30
Random Seed                  : 42
======================================================================
  GA configuration initialized successfully.
```

[80]:
```
# CELL 33: Chromosome Representation

#
# Each chromosome represents one complete feasible UAV route
# from the Start node to the Goal node.
#
# The chromosome stores:
#
#   • Path
#   • Total traversal cost
#   • Fitness
#
# Cost evaluation reuses the existing Grid class and
# movement_cost() function already implemented.
#


from dataclasses import dataclass, field
from typing import List, Tuple

Coordinate = Tuple[int, int]
```

```python
# Chromosome


@dataclass
class Chromosome:
    """
    Represents one candidate UAV route.

    Attributes
    ----------
    path : list
        Complete route from start to goal.

    total_cost : float
        Sum of movement costs.

    fitness : float
        Fitness value used by the GA.
    """

    path: List[Coordinate]

    total_cost: float = field(default=float("inf"))

    fitness: float = field(default=0.0)


    # Evaluate Cost


    def evaluate(
        self,
        grid
    ) -> float:
        """
        Compute total route cost using the existing
        movement_cost() function.
        """

        cost = 0.0

        for i in range(len(self.path) - 1):

            cost += grid.movement_cost(
                self.path[i],
                self.path[i + 1]
            )
```

```
        self.total_cost = cost

        return cost


    # Compute Fitness


    def compute_fitness(self) -> float:
        """
        Fitness function

            Fitness = 1 / (1 + Cost)

        Higher fitness corresponds to lower cost.
        """

        self.fitness = 1.0 / (1.0 + self.total_cost)

        return self.fitness


    # Evaluate Chromosome


    def evaluate_fitness(
        self,
        grid
    ) -> float:
        """
        Evaluate both cost and fitness.
        """

        self.evaluate(grid)

        self.compute_fitness()

        return self.fitness


    # Clone


    def copy(self):
        """
        Deep copy of chromosome.
```

```
        """

        return Chromosome(
            path=self.path.copy(),
            total_cost=self.total_cost,
            fitness=self.fitness
        )


    # Path Length


    @property
    def length(self):

        return len(self.path)


    # String Representation


    def __repr__(self):

        return (

            f"Chromosome("

            f"length={len(self.path)}, "

            f"cost={self.total_cost:.4f}, "

            f"fitness={self.fitness:.6f}"

            f")"

        )



# Simple Test (Optional)


print("=" * 65)
print("GENETIC ALGORITHM CHROMOSOME")
print("=" * 65)

print("Chromosome class successfully initialized.")
```

```
print("=" * 65)
```

```
=================================================================
GENETIC ALGORITHM CHROMOSOME
=================================================================
Chromosome class successfully initialized.
=================================================================
```

[81]:
```
# CELL 34: Initial Population Generation

#
# Generates an initial population of feasible chromosomes.
#
# Strategy
# --------
# 1. Seed the population using the existing D* Lite solution.
# 2. Generate additional feasible paths using randomized walks.
# 3. Evaluate every chromosome.
#
# This implementation reuses the existing Grid helper methods:
#
#   • grid.neighbors()
#   • grid.is_valid()
#   • grid.movement_cost()
#


import random
from typing import List, Optional


class PopulationInitializer:
    """
    Generates the initial GA population.
    """

    def __init__(self, grid):

        self.grid = grid


    # Generate Random Feasible Path


    def random_path(
        self,
```

```
        start,
        goal,
        max_steps: int = 500
    ) -> Optional[List[Coordinate]]:
        """
        Generate one feasible path using randomized expansion.
        """

        current = start

        path = [current]

        visited = {current}

        steps = 0

        while current != goal and steps < max_steps:

            neighbors = self.grid.neighbors(current)

            # Avoid revisiting nodes whenever possible
            candidates = [

                node

                for node in neighbors

                if node not in visited

            ]

            if not candidates:

                candidates = neighbors

            if len(candidates) == 0:

                return None

            # Prefer nodes closer to goal
            candidates.sort(

                key=lambda node:
                self.grid.distance(node, goal)

            )
```

```
            # Randomly choose among best few candidates
            k = min(3, len(candidates))

            current = random.choice(candidates[:k])

            path.append(current)

            visited.add(current)

            steps += 1

        if current != goal:

            return None

        return path


    # Generate Population


    def generate_population(
        self,
        start,
        goal,
        seed_path: Optional[List[Coordinate]],
        config: GAConfig
    ) -> List[Chromosome]:

        population = []


        # Seed using D* Lite solution


        if seed_path is not None:

            # chromosome = Chromosome(seed_path.copy())
            if isinstance(seed_path, tuple):
                seed_path = seed_path[0]

            chromosome = Chromosome(list(seed_path))

            chromosome.evaluate_fitness(self.grid)

            population.append(chromosome)
```

```
        # Random Individuals

        attempts = 0
        while (
            len(population) < config.population_size
            and
            attempts < config.max_initialization_attempts
        ):
            candidate = self.random_path(
                start,
                goal
            )
            attempts += 1
            if candidate is None:
                continue
            chromosome = Chromosome(candidate)
            chromosome.evaluate_fitness(self.grid)
            population.append(chromosome)

        # Sort Population

        population.sort(
            key=lambda c: c.total_cost
        )
        return population
```

```
# Example Usage


print("=" * 70)
print("INITIAL POPULATION INITIALIZER")
print("=" * 70)

population_initializer = PopulationInitializer(grid)

print("Population initializer ready.")

print("  D* Lite path will be used as seed chromosome.")

print("=" * 70)
```

```
======================================================================
INITIAL POPULATION INITIALIZER
======================================================================
Population initializer ready.
  D* Lite path will be used as seed chromosome.
======================================================================
```

[82]:
```
# ============================================================
# CELL 35: Population Evaluation & Tournament Selection
# ============================================================
#
# This cell provides utility functions for:
#
# • Evaluating an entire population
# • Retrieving the best chromosome
# • Computing average fitness
# • Tournament Selection
#
# These functions will be reused by the GeneticPlanner.
#
# ============================================================

import random
from typing import List


class GAOperators:
    """
    Collection of Genetic Algorithm utility operators.
```

```
    """


    # Evaluate Population


    @staticmethod
    def evaluate_population(
        population: List[Chromosome],
        grid
    ) -> None:
        """
        Evaluate cost and fitness of every chromosome.
        """

        for chromosome in population:

            chromosome.evaluate_fitness(grid)


    # Best Chromosome


    @staticmethod
    def best_chromosome(
        population: List[Chromosome]
    ) -> Chromosome:
        """
        Return chromosome with lowest total cost.
        """

        return min(
            population,
            key=lambda c: c.total_cost
        )


    # Average Fitness


    @staticmethod
    def average_fitness(
        population: List[Chromosome]
    ) -> float:
        """
        Compute average population fitness.
        """
```

```
        if len(population) == 0:
            return 0.0

        return sum(
            chromosome.fitness
            for chromosome in population
        ) / len(population)


    # Tournament Selection


    @staticmethod
    def tournament_selection(
        population: List[Chromosome],
        tournament_size: int
    ) -> Chromosome:
        """
        Select one parent using tournament selection.

        The chromosome with the lowest cost wins.
        """

        tournament = random.sample(
            population,
            k=min(tournament_size, len(population))
        )

        winner = min(
            tournament,
            key=lambda chromosome: chromosome.total_cost
        )

        # Return a copy so the original remains unchanged
        return winner.copy()


    # Elite Selection


    @staticmethod
    def elite_selection(
        population: List[Chromosome],
        elite_count: int
    ) -> List[Chromosome]:
        """
```

```
        Preserve the best chromosomes.
        """

        population = sorted(
            population,
            key=lambda c: c.total_cost
        )

        return [
            chromosome.copy()
            for chromosome in population[:elite_count]
        ]


# ============================================================
# Example Test
# ============================================================

print("=" * 65)
print("GA OPERATORS INITIALIZED")
print("=" * 65)

print("  Population Evaluation")
print("  Tournament Selection")
print("  Elite Selection")
print("  Best Chromosome Retrieval")
print("  Average Fitness Calculation")

print("=" * 65)
```

```
=================================================================
GA OPERATORS INITIALIZED
=================================================================
  Population Evaluation
  Tournament Selection
  Elite Selection
  Best Chromosome Retrieval
  Average Fitness Calculation
=================================================================
```

[83]:
```
# CELL 36: Path-Based Crossover

#
# This cell implements crossover for two parent paths.
#
# Strategy
# --------
```

```
# 1. Find common nodes shared by both parents.
# 2. Randomly select one common node as crossover point.
# 3. Exchange path segments.
# 4. Remove duplicate loops.
# 5. If crossover is not possible, return copies of parents.
#
# This operator preserves:
#   • Start node
#   • Goal node
#   • Feasible ordering of nodes
#


from typing import List, Tuple
import random


class GACrossover:
    """
    Path-based crossover operator.
    """


    # Remove loops from a path


    @staticmethod
    def remove_loops(
        path: List[Coordinate]
    ) -> List[Coordinate]:
        """
        Removes repeated cycles while preserving path order.

        Example

        A B C D C E

        becomes

        A B C E
        """

        new_path = []
        visited = {}

        for node in path:
```

```
            if node in visited:

                index = visited[node]

                new_path = new_path[:index + 1]

                visited = {
                    n: i
                    for i, n in enumerate(new_path)
                }

            else:

                visited[node] = len(new_path)

                new_path.append(node)

        return new_path


    # Perform crossover


    @staticmethod
    def crossover(
        parent1: Chromosome,
        parent2: Chromosome
    ) -> Tuple[Chromosome, Chromosome]:
        """
        Path-based crossover using common waypoints.

        Returns
        -------
        offspring1
        offspring2
        """

        path1 = parent1.path
        path2 = parent2.path


        # Find common nodes (excluding start and goal)


        common_nodes = list(

            set(path1[1:-1])
```

```
        &
        set(path2[1:-1])
    )

    # No common waypoint
    if len(common_nodes) == 0:
        return (
            parent1.copy(),
            parent2.copy()
        )

    # Random crossover node
    crossover_node = random.choice(common_nodes)
    idx1 = path1.index(crossover_node)
    idx2 = path2.index(crossover_node)

    # Exchange tails
    child1_path = (
        path1[:idx1]
        +
        path2[idx2:]
    )
    child2_path = (
```

```
            path2[:idx2]

            +

            path1[idx1:]

        )


        # Remove cycles


        child1_path = GACrossover.remove_loops(

            child1_path

        )

        child2_path = GACrossover.remove_loops(

            child2_path

        )

        offspring1 = Chromosome(child1_path)

        offspring2 = Chromosome(child2_path)

        return offspring1, offspring2



# Example Initialization


print("=" * 65)
print("GA CROSSOVER INITIALIZED")
print("=" * 65)

print("  Path-based crossover")
print("  Loop removal")
print("  Common waypoint crossover")
print("  Parent preservation when crossover fails")

print("=" * 65)
```

==========

```
GA CROSSOVER INITIALIZED
================================================================
 Path-based crossover
 Loop removal
 Common waypoint crossover
 Parent preservation when crossover fails
================================================================
```

[84]:
```python
# CELL 37: Mutation and Path Repair

#
# Mutation introduces diversity into the population by
# modifying a small section of an existing path.
#
# Strategy
# --------
# 1. Randomly choose an intermediate waypoint.
# 2. Replace it with one of its valid neighboring cells.
# 3. Verify connectivity.
# 4. Remove loops if introduced.
# 5. Re-evaluate chromosome.
#
# Since the mutation is local, feasibility is usually
# preserved while encouraging exploration.
#


import random


class GAMutation:
    """
    Mutation and path repair operators.
    """


    # Check path validity


    @staticmethod
    def is_path_valid(
        path,
        grid
    ):
        """
        Verify every node is valid and every consecutive
        pair of nodes are neighbors.
```

```
        """

        if len(path) < 2:
            return False

        for node in path:

            if not grid.is_valid(node):
                return False

        for i in range(len(path)-1):

            if path[i+1] not in grid.neighbors(path[i]):
                return False

        return True


    # Remove duplicate loops


    @staticmethod
    def repair_path(path):
        """
        Remove repeated cycles while preserving order.
        """

        repaired = []
        visited = {}

        for node in path:

            if node in visited:

                idx = visited[node]

                repaired = repaired[:idx+1]

                visited = {
                    n:i
                    for i,n in enumerate(repaired)
                }

            else:

                visited[node] = len(repaired)
```

```
                repaired.append(node)

        return repaired


    # Mutation


    @staticmethod
    def mutate(
        chromosome,
        grid,
        mutation_rate
    ):
        """
        Local waypoint mutation.
        """

        child = chromosome.copy()

        # No mutation
        if random.random() > mutation_rate:
            return child

        # Need at least start-mid-goal
        if len(child.path) <= 2:
            return child

        # Random intermediate waypoint
        index = random.randint(
            1,
            len(child.path)-2
        )

        current = child.path[index]

        neighbors = grid.neighbors(current)

        # Remove invalid choices
        candidates = []

        for node in neighbors:

            prev_node = child.path[index-1]
            next_node = child.path[index+1]

            if (
```

```
                node in grid.neighbors(prev_node)
                and
                next_node in grid.neighbors(node)
            ):

                candidates.append(node)

        if len(candidates) == 0:
            return child

        # Replace waypoint
        child.path[index] = random.choice(candidates)

        # Repair loops
        child.path = GAMutation.repair_path(
            child.path
        )

        # Verify validity
        if not GAMutation.is_path_valid(
            child.path,
            grid
        ):
            return chromosome.copy()

        child.evaluate_fitness(grid)

        return child


print("="*65)
print("GA MUTATION INITIALIZED")
print("="*65)

print("  Local waypoint mutation")
print("  Connectivity checking")
print("  Path repair")
print("  Loop removal")
print("  Path validity verification")

print("="*65)
```

```
=================================================================
GA MUTATION INITIALIZED
=================================================================
  Local waypoint mutation
  Connectivity checking
```

```
 Path repair
 Loop removal
 Path validity verification
================================================================
```

[85]:
```python
# CELL 38: GeneticPlanner

#
# This class implements the complete Genetic Algorithm planner.
#
# Workflow
# --------
# 1. Generate initial population
# 2. Evaluate population
# 3. Preserve elite chromosomes
# 4. Tournament selection
# 5. Crossover
# 6. Mutation
# 7. Repeat for specified generations
# 8. Return best path
#
# This planner is independent of D* Lite while reusing the
# same Grid environment and cost functions.
#


import time
from typing import List, Tuple


class GeneticPlanner:
    """
    Genetic Algorithm based UAV Route Planner.
    """

    def __init__(
        self,
        grid,
        config: GAConfig = GAConfig()
    ):

        self.grid = grid
        self.config = config


    # Main Planning Function
```

```
def plan(
    self,
    start: Coordinate,
    goal: Coordinate,
    seed_path: List[Coordinate] | None = None
) -> Tuple[List[Coordinate], float]:

    start_time = time.perf_counter()


    # Initial Population


    initializer = PopulationInitializer(self.grid)

    population = initializer.generate_population(

        start=start,

        goal=goal,

        seed_path=seed_path,

        config=self.config

    )

    if len(population) == 0:

        raise RuntimeError(
            "Unable to generate an initial population."
        )

    # Evaluate
    GAOperators.evaluate_population(
        population,
        self.grid
    )


    # Evolution Loop


    for generation in range(self.config.generations):

        # ---------------- Elite --------------------
```

```
        new_population = GAOperators.elite_selection(
            population,
            self.config.elite_count
        )
        # ------------ Generate Children ------------
        while len(new_population) < self.config.population_size:
            parent1 = GAOperators.tournament_selection(
                population,
                self.config.tournament_size
            )
            parent2 = GAOperators.tournament_selection(
                population,
                self.config.tournament_size
            )
            # ----------- Crossover -------------
            if random.random() < self.config.crossover_rate:
                child1, child2 = GACrossover.crossover(
                    parent1,
                    parent2
                )
            else:
                child1 = parent1.copy()
                child2 = parent2.copy()
```

```
            # ------------ Mutation -------------

            child1 = GAMutation.mutate(

                child1,

                self.grid,

                self.config.mutation_rate

            )

            child2 = GAMutation.mutate(

                child2,

                self.grid,

                self.config.mutation_rate

            )

            child1.evaluate_fitness(self.grid)
            child2.evaluate_fitness(self.grid)

            new_population.append(child1)

            if len(new_population) < self.config.population_size:

                new_population.append(child2)


        population = sorted(

            new_population,

            key=lambda c: c.total_cost

        )


        # Statistics


        best = GAOperators.best_chromosome(population)
```

```
            avg_fit = GAOperators.average_fitness(population)

            print(
                f"Generation "
                f"{generation+1:03d} | "
                f"Best Cost = {best.total_cost:.4f} | "
                f"Best Fitness = {best.fitness:.6f} | "
                f"Average Fitness = {avg_fit:.6f}"
            )


        # Final Result


        best = GAOperators.best_chromosome(population)

        execution_time = time.perf_counter() - start_time

        print("\n" + "="*70)

        print("GENETIC ALGORITHM COMPLETED")

        print("="*70)

        print(f"Generations      : {self.config.generations}")

        print(f"Population Size  : {self.config.population_size}")

        print(f"Best Route Cost  : {best.total_cost:.4f}")

        print(f"Fitness          : {best.fitness:.6f}")

        print(f"Execution Time   : {execution_time:.4f} sec")

        print("="*70)

        return best.path, best.total_cost
```

[86]:
```
print(type(planner.reconstruct_path()))
print(planner.reconstruct_path())
```

```
<class 'tuple'>
([(2, 26), (3, 26), (4, 26), (5, 26), (6, 26), (7, 26), (8, 26), (9, 26), (10,
26), (11, 26), (12, 26), (13, 26), (14, 26), (15, 26), (16, 26), (17, 26), (18,
26), (19, 26), (20, 26), (21, 26), (22, 26), (23, 26), (24, 26), (25, 26), (26,
26), (27, 26), (28, 26), (29, 26), (30, 26), (31, 26), (32, 26), (33, 26), (34,
26), (35, 26), (36, 26), (37, 26), (38, 26), (39, 26), (40, 26), (41, 26), (42,
```

```
26), (43, 26), (44, 26), (45, 26), (46, 26), (47, 26), (48, 26), (49, 26), (50,
26), (51, 26), (52, 26), (53, 26), (54, 26), (55, 26), (56, 26), (57, 26), (58,
26), (59, 26), (60, 26), (61, 26), (62, 26), (63, 26), (64, 26), (65, 26), (66,
26), (67, 26), (68, 26), (69, 26), (70, 26), (71, 26), (72, 26), (73, 26), (74,
26), (75, 27), (76, 28), (77, 29), (78, 28), (79, 27), (80, 27), (81, 27), (82,
27), (83, 27), (84, 26), (85, 26), (86, 26), (87, 26)], 87.48528137423855,
np.float64(4.080391164923183))
```

[87]:
```python
# CELL 39: Planner Selection and Execution

#
# This cell allows the user to choose between
#
#   1. D* Lite
#   2. Genetic Algorithm
#
# Both planners return
#
#   path
#   total_cost
#
# so that all subsequent mission metrics and visualization
# cells can be reused without modification.
#


import time


# Select Planner


# Available options:
#
# "DSTAR"
# "GA"

selected_planner = "GA"

print("=" * 70)
print("PLANNER CONFIGURATION")
print("=" * 70)

print(f"Selected Planner : {selected_planner}")

print("=" * 70)
```

```
# Execute Planner


planning_start = time.perf_counter()

if selected_planner.upper() == "DSTAR":

    print("\nExecuting D* Lite Planner...\n")

    # Assumes your D* Lite planner has already been created
    # earlier in the notebook.
    #
    # Example:
    #
    # planner = DStarLite(grid, start, goal)
    #
    # Modify this line only if your planner object has a
    # different variable name.

    path, mission_distance, total_cost = planner.plan()

elif selected_planner.upper() == "GA":

    print("\nExecuting Genetic Algorithm Planner...\n")

    ga_planner = GeneticPlanner(

        grid=grid,

        config=ga_config

    )


    # Seed with D* Lite solution if available


    try:

        seed_path = planner.reconstruct_path()

        print("  Using D* Lite solution as seed.")

    except Exception:

        seed_path = None
```

```
        print("No D* Lite seed available.")
    path, total_cost = ga_planner.plan(
        start=start,
        goal=goal,
        seed_path=seed_path
    )

    # Compute Total Distance

    mission_distance = 0.0
    for i in range(len(path) - 1):
        mission_distance += grid.distance(
            path[i],
            path[i + 1]
        )
else:
    raise ValueError(
        "Planner must be either 'DSTAR' or 'GA'."
    )
planning_end = time.perf_counter()
execution_time = planning_end - planning_start

# Display Results

print("\n" + "=" * 70)
```

```
print("PLANNING COMPLETED")

print("=" * 70)

print(f"Planner           : {selected_planner}")

print(f"Waypoints         : {len(path)}")

print(f"Mission Distance  : {mission_distance:.2f}")

print(f"Total Cost        : {total_cost:.4f}")

print(f"Execution Time    : {execution_time:.4f} sec")

print("=" * 70)


# Store Results


planner_results = {

    "Planner": selected_planner,

    "Path": path,

    "Distance": mission_distance,

    "Cost": total_cost,

    "Execution Time": execution_time

}

print("  Planner results stored successfully.")
```

```
======================================================================
PLANNER CONFIGURATION
======================================================================
Selected Planner : GA
======================================================================

Executing Genetic Algorithm Planner…

  Using D* Lite solution as seed.
Generation 001 | Best Cost = 3.1993 | Best Fitness = 0.238133 | Average Fitness
= 0.191777
```

```
Generation 002 | Best Cost = 2.9224 | Best Fitness = 0.254944 | Average Fitness
= 0.218093
Generation 003 | Best Cost = 2.9224 | Best Fitness = 0.254944 | Average Fitness
= 0.238434
Generation 004 | Best Cost = 2.9136 | Best Fitness = 0.255517 | Average Fitness
= 0.249983
Generation 005 | Best Cost = 2.9136 | Best Fitness = 0.255517 | Average Fitness
= 0.254773
Generation 006 | Best Cost = 2.9136 | Best Fitness = 0.255517 | Average Fitness
= 0.255159
Generation 007 | Best Cost = 2.9136 | Best Fitness = 0.255517 | Average Fitness
= 0.255417
Generation 008 | Best Cost = 2.9136 | Best Fitness = 0.255517 | Average Fitness
= 0.255517
Generation 009 | Best Cost = 2.9009 | Best Fitness = 0.256354 | Average Fitness
= 0.255494
Generation 010 | Best Cost = 2.9009 | Best Fitness = 0.256354 | Average Fitness
= 0.255643
Generation 011 | Best Cost = 2.9009 | Best Fitness = 0.256354 | Average Fitness
= 0.255476
Generation 012 | Best Cost = 2.9009 | Best Fitness = 0.256354 | Average Fitness
= 0.255866
Generation 013 | Best Cost = 2.8542 | Best Fitness = 0.259458 | Average Fitness
= 0.256432
Generation 014 | Best Cost = 2.8542 | Best Fitness = 0.259458 | Average Fitness
= 0.256488
Generation 015 | Best Cost = 2.8542 | Best Fitness = 0.259458 | Average Fitness
= 0.256922
Generation 016 | Best Cost = 2.8542 | Best Fitness = 0.259458 | Average Fitness
= 0.258266
Generation 017 | Best Cost = 2.8375 | Best Fitness = 0.260586 | Average Fitness
= 0.259243
Generation 018 | Best Cost = 2.7882 | Best Fitness = 0.263975 | Average Fitness
= 0.259656
Generation 019 | Best Cost = 2.7468 | Best Fitness = 0.266892 | Average Fitness
= 0.260327
Generation 020 | Best Cost = 2.6809 | Best Fitness = 0.271673 | Average Fitness
= 0.262259
Generation 021 | Best Cost = 2.6642 | Best Fitness = 0.272909 | Average Fitness
= 0.265881
Generation 022 | Best Cost = 2.6610 | Best Fitness = 0.273147 | Average Fitness
= 0.269236
Generation 023 | Best Cost = 2.6610 | Best Fitness = 0.273147 | Average Fitness
= 0.272134
Generation 024 | Best Cost = 2.6610 | Best Fitness = 0.273147 | Average Fitness
= 0.272991
Generation 025 | Best Cost = 2.6610 | Best Fitness = 0.273147 | Average Fitness
= 0.273015
```

```
Generation 026 | Best Cost = 2.6610 | Best Fitness = 0.273147 | Average Fitness
= 0.273021
Generation 027 | Best Cost = 2.6610 | Best Fitness = 0.273147 | Average Fitness
= 0.273138
Generation 028 | Best Cost = 2.6610 | Best Fitness = 0.273147 | Average Fitness
= 0.273147
Generation 029 | Best Cost = 2.6517 | Best Fitness = 0.273843 | Average Fitness
= 0.273068
Generation 030 | Best Cost = 2.6031 | Best Fitness = 0.277540 | Average Fitness
= 0.273223
Generation 031 | Best Cost = 2.6031 | Best Fitness = 0.277540 | Average Fitness
= 0.273796
Generation 032 | Best Cost = 2.5938 | Best Fitness = 0.278258 | Average Fitness
= 0.276029
Generation 033 | Best Cost = 2.5938 | Best Fitness = 0.278258 | Average Fitness
= 0.277737
Generation 034 | Best Cost = 2.5512 | Best Fitness = 0.281597 | Average Fitness
= 0.278144
Generation 035 | Best Cost = 2.5512 | Best Fitness = 0.281597 | Average Fitness
= 0.278679
Generation 036 | Best Cost = 2.5363 | Best Fitness = 0.282783 | Average Fitness
= 0.279978
Generation 037 | Best Cost = 2.5363 | Best Fitness = 0.282783 | Average Fitness
= 0.281537
Generation 038 | Best Cost = 2.5258 | Best Fitness = 0.283627 | Average Fitness
= 0.281212
Generation 039 | Best Cost = 2.5258 | Best Fitness = 0.283627 | Average Fitness
= 0.282803
Generation 040 | Best Cost = 2.5258 | Best Fitness = 0.283627 | Average Fitness
= 0.283226
Generation 041 | Best Cost = 2.5023 | Best Fitness = 0.285528 | Average Fitness
= 0.283674
Generation 042 | Best Cost = 2.5023 | Best Fitness = 0.285528 | Average Fitness
= 0.283816
Generation 043 | Best Cost = 2.5023 | Best Fitness = 0.285528 | Average Fitness
= 0.284329
Generation 044 | Best Cost = 2.5023 | Best Fitness = 0.285528 | Average Fitness
= 0.285290
Generation 045 | Best Cost = 2.5023 | Best Fitness = 0.285528 | Average Fitness
= 0.285367
Generation 046 | Best Cost = 2.5023 | Best Fitness = 0.285528 | Average Fitness
= 0.285528
Generation 047 | Best Cost = 2.5023 | Best Fitness = 0.285528 | Average Fitness
= 0.285521
Generation 048 | Best Cost = 2.4687 | Best Fitness = 0.288290 | Average Fitness
= 0.285590
Generation 049 | Best Cost = 2.4687 | Best Fitness = 0.288290 | Average Fitness
= 0.285873
```

```
Generation 050 | Best Cost = 2.4687 | Best Fitness = 0.288290 | Average Fitness
= 0.287090
Generation 051 | Best Cost = 2.4687 | Best Fitness = 0.288290 | Average Fitness
= 0.287996
Generation 052 | Best Cost = 2.4687 | Best Fitness = 0.288290 | Average Fitness
= 0.288289
Generation 053 | Best Cost = 2.4687 | Best Fitness = 0.288290 | Average Fitness
= 0.287788
Generation 054 | Best Cost = 2.4267 | Best Fitness = 0.291824 | Average Fitness
= 0.288074
Generation 055 | Best Cost = 2.3820 | Best Fitness = 0.295684 | Average Fitness
= 0.288545
Generation 056 | Best Cost = 2.3820 | Best Fitness = 0.295684 | Average Fitness
= 0.290511
Generation 057 | Best Cost = 2.3820 | Best Fitness = 0.295684 | Average Fitness
= 0.293890
Generation 058 | Best Cost = 2.3820 | Best Fitness = 0.295684 | Average Fitness
= 0.295666
Generation 059 | Best Cost = 2.3820 | Best Fitness = 0.295684 | Average Fitness
= 0.295684
Generation 060 | Best Cost = 2.3820 | Best Fitness = 0.295685 | Average Fitness
= 0.295457
Generation 061 | Best Cost = 2.3717 | Best Fitness = 0.296584 | Average Fitness
= 0.295632
Generation 062 | Best Cost = 2.3717 | Best Fitness = 0.296584 | Average Fitness
= 0.295881
Generation 063 | Best Cost = 2.3619 | Best Fitness = 0.297449 | Average Fitness
= 0.296219
Generation 064 | Best Cost = 2.3619 | Best Fitness = 0.297449 | Average Fitness
= 0.296655
Generation 065 | Best Cost = 2.3555 | Best Fitness = 0.298014 | Average Fitness
= 0.296936
Generation 066 | Best Cost = 2.3555 | Best Fitness = 0.298014 | Average Fitness
= 0.297437
Generation 067 | Best Cost = 2.3424 | Best Fitness = 0.299186 | Average Fitness
= 0.297659
Generation 068 | Best Cost = 2.3424 | Best Fitness = 0.299186 | Average Fitness
= 0.297952
Generation 069 | Best Cost = 2.3424 | Best Fitness = 0.299186 | Average Fitness
= 0.298629
Generation 070 | Best Cost = 2.3424 | Best Fitness = 0.299186 | Average Fitness
= 0.299098
Generation 071 | Best Cost = 2.3424 | Best Fitness = 0.299186 | Average Fitness
= 0.299156
Generation 072 | Best Cost = 2.3313 | Best Fitness = 0.300182 | Average Fitness
= 0.299018
Generation 073 | Best Cost = 2.3229 | Best Fitness = 0.300941 | Average Fitness
= 0.299329
```

```
Generation 074 | Best Cost = 2.3229 | Best Fitness = 0.300941 | Average Fitness
= 0.299468
Generation 075 | Best Cost = 2.3229 | Best Fitness = 0.300941 | Average Fitness
= 0.299638
Generation 076 | Best Cost = 2.3229 | Best Fitness = 0.300942 | Average Fitness
= 0.300121
Generation 077 | Best Cost = 2.3057 | Best Fitness = 0.302507 | Average Fitness
= 0.300856
Generation 078 | Best Cost = 2.3057 | Best Fitness = 0.302507 | Average Fitness
= 0.301176
Generation 079 | Best Cost = 2.2969 | Best Fitness = 0.303313 | Average Fitness
= 0.301841
Generation 080 | Best Cost = 2.2903 | Best Fitness = 0.303923 | Average Fitness
= 0.302492
Generation 081 | Best Cost = 2.2815 | Best Fitness = 0.304736 | Average Fitness
= 0.303194
Generation 082 | Best Cost = 2.2815 | Best Fitness = 0.304736 | Average Fitness
= 0.303129
Generation 083 | Best Cost = 2.2727 | Best Fitness = 0.305559 | Average Fitness
= 0.304441
Generation 084 | Best Cost = 2.2727 | Best Fitness = 0.305559 | Average Fitness
= 0.304824
Generation 085 | Best Cost = 2.2727 | Best Fitness = 0.305559 | Average Fitness
= 0.305036
Generation 086 | Best Cost = 2.2727 | Best Fitness = 0.305560 | Average Fitness
= 0.305506
Generation 087 | Best Cost = 2.2727 | Best Fitness = 0.305560 | Average Fitness
= 0.305462
Generation 088 | Best Cost = 2.2727 | Best Fitness = 0.305560 | Average Fitness
= 0.305560
Generation 089 | Best Cost = 2.2727 | Best Fitness = 0.305560 | Average Fitness
= 0.305560
Generation 090 | Best Cost = 2.2727 | Best Fitness = 0.305561 | Average Fitness
= 0.305560
Generation 091 | Best Cost = 2.2727 | Best Fitness = 0.305562 | Average Fitness
= 0.305365
Generation 092 | Best Cost = 2.2727 | Best Fitness = 0.305562 | Average Fitness
= 0.305561
Generation 093 | Best Cost = 2.2727 | Best Fitness = 0.305562 | Average Fitness
= 0.305562
Generation 094 | Best Cost = 2.2727 | Best Fitness = 0.305562 | Average Fitness
= 0.305562
Generation 095 | Best Cost = 2.2727 | Best Fitness = 0.305562 | Average Fitness
= 0.305288
Generation 096 | Best Cost = 2.2726 | Best Fitness = 0.305570 | Average Fitness
= 0.305543
Generation 097 | Best Cost = 2.2726 | Best Fitness = 0.305570 | Average Fitness
= 0.305301
```

```
Generation 098 | Best Cost = 2.2726 | Best Fitness = 0.305570 | Average Fitness
= 0.305565
Generation 099 | Best Cost = 2.2640 | Best Fitness = 0.306369 | Average Fitness
= 0.305589
Generation 100 | Best Cost = 2.2488 | Best Fitness = 0.307804 | Average Fitness
= 0.305183

======================================================================
GENETIC ALGORITHM COMPLETED
======================================================================
Generations      : 100
Population Size  : 40
Best Route Cost  : 2.2488
Fitness          : 0.307804
Execution Time   : 0.6625 sec
======================================================================

======================================================================
PLANNING COMPLETED
======================================================================
Planner          : GA
Waypoints        : 89
Mission Distance  : 109.12
Total Cost       : 2.2488
Execution Time    : 0.6636 sec
======================================================================
 Planner results stored successfully.
```

[88]:
```python
# CELL 40: Planner Comparison

#
# Compare the performance of
#
#   • D* Lite
#   • Genetic Algorithm
#
# using identical start, goal and environment.
#


comparison_results = pd.DataFrame({

    "Planner": [

        "D* Lite",
        "Genetic Algorithm"
```

```
    ],

    "Route Cost": [

        dstar_total_cost,
        total_cost

    ],

    "Distance": [

        dstar_distance,
        mission_distance

    ],

    "Execution Time (s)": [

        dstar_execution_time,
        execution_time

    ],

    "Waypoints": [

        len(dstar_path),
        len(path)

    ]

})

print("="*70)
print("PLANNER COMPARISON")
print("="*70)

display(comparison_results)


# Determine Better Planner


best_cost = comparison_results.loc[
    comparison_results["Route Cost"].idxmin()
]

fastest = comparison_results.loc[
```

```
    comparison_results["Execution Time (s)"].idxmin()
]

print("\nBest Cost Planner")
print("-----------------")
print(best_cost["Planner"])

print("\nFastest Planner")
print("----------------")
print(fastest["Planner"])
```

```
======================================================================
PLANNER COMPARISON
======================================================================

             Planner  Route Cost  Distance  Execution Time (s)  Waypoints
0            D* Lite        4.08     87.49            5.08e-03         86
1  Genetic Algorithm        2.25    109.12            6.64e-01         89


Best Cost Planner
-----------------
Genetic Algorithm

Fastest Planner
---------------
D* Lite
```

[88]: